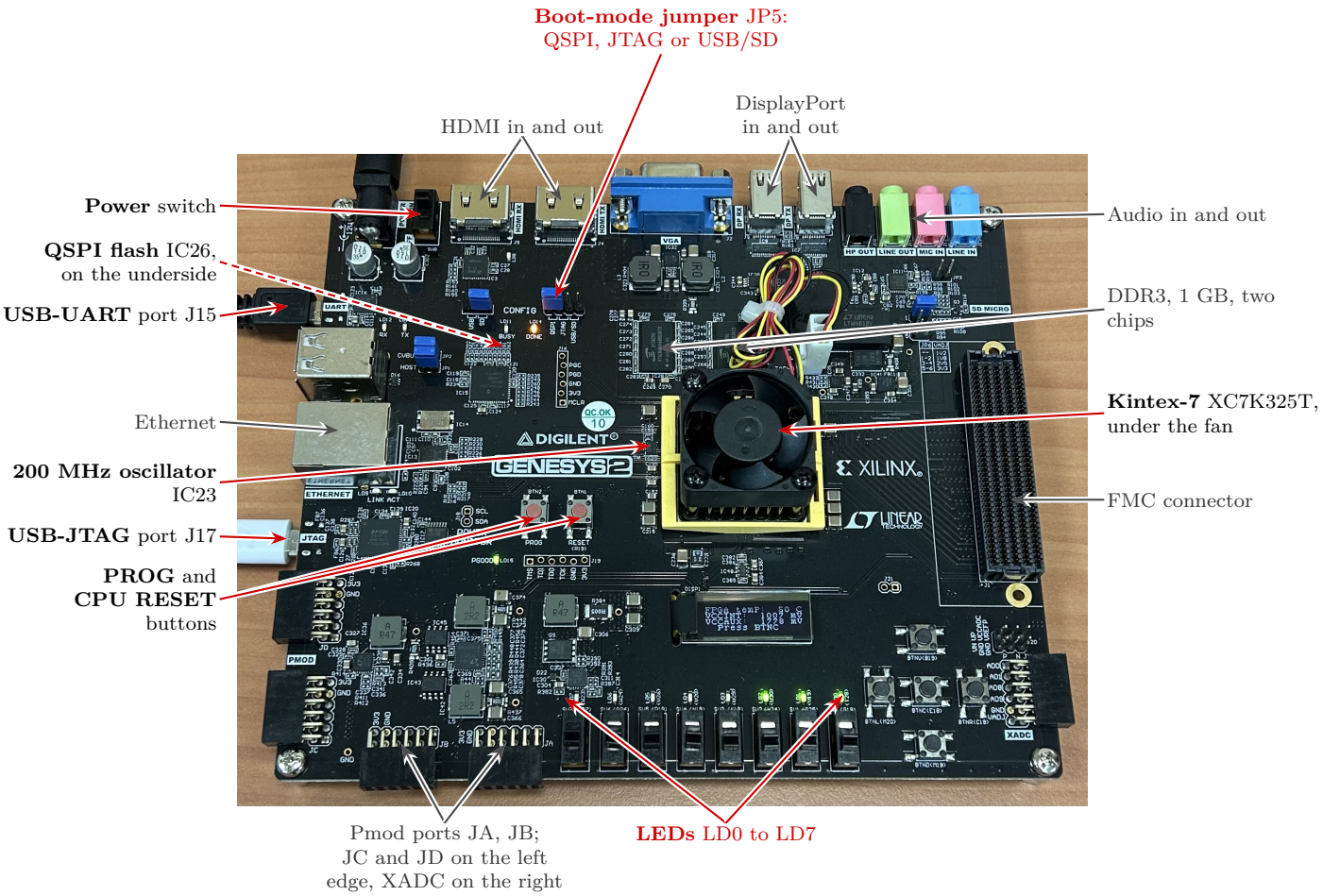
A Multi-cycle RV32I System on Chip for the Digilent Genesys 2

Amit Malach

Supervisor: Prof. Avi Mendelson
Technion – Israel Institute of Technology

Final report, October 28, 2026



The Digilent Genesys 2, powered on and running Digilent's factory demo. In **red**, the parts this design uses: the FPGA, its clock, the two USB ports that program it and carry its output, the flash it boots from, the LEDs and the buttons. In grey, the parts it leaves unused; Chapter 12 ranks what they could add. The dashed line points through the board.

Abstract

This report describes a small system on chip built around a multi-cycle RV32I core, written in Verilog and run at 200 MHz on the Kintex-7 FPGA of a Digilent Genesys 2 board. The core executes the RV32I base integer instruction set, one instruction at a time, in four to six clock cycles each, under a seven-state finite state machine. Around it sit 6 KB of block RAM, a memory-mapped LED register, a UART transmitter and a controller for the board's 32 MB QSPI flash, all on one bus that the core sees as plain memory. Two boot modes share the same hardware. In BRAM boot the firmware image is part of the bitstream and the program runs from the RAM; in flash boot the core executes a program in place from the flash, which admits programs larger than the RAM at about a hundredth of the speed.

The design follows Bruno Levy's *From Blinker to RISC-V*. It is checked at three levels: an ALU bench against values worked out by hand, a golden trace of the mandelbrot demo's UART output and cycle count, and a lint of the repository's conventions; each is a make target whose log a commit must cite. The whole SoC takes 984 look-up tables and 244 flip-flops, under half a percent of the device, meets its 5 ns clock period with 0.219 ns to spare, and is estimated to draw 35 mW beyond the device's static power.

On the board, the mandelbrot demo's first frame is byte for byte the golden trace's, which is in turn what the C source prints when compiled for a PC, and its frame time of 1.03 s matches the simulated cycle count to 0.2 %. The tinyraytracer demo, five times the RAM's size, ran in flash boot. The report records the pitfalls met on the way, from the board's UART net names to the flash clock's hand-over after configuration, and ranks the extensions the next student could take on: pipelining, the M extension, UART receive, interrupts, DDR3 and HDMI. Every figure that a script can make is regenerated by a make target from the repository, and the raw board captures are committed as evidence.

Contents

1 Introduction **1**
- 1.1 The project . . . 1
- 1.2 What was built . . . 2
- 1.3 Results in brief . . . 2
- 1.4 How to read this report . . . 3

2 Board and tools **4**
- 2.1 Tools . . . 6

3 Repository tour **8**

4 System architecture **10**
- 4.1 The bus and the modules . . . 10
- 4.2 Memory map and address decode . . . 11
- 4.3 Clock and reset . . . 12

5 The multi-cycle core **13**
- 5.1 The state machine . . . 13
- 5.2 The datapath . . . 14
- 5.3 One instruction through the states . . . 15

6 Memory and memory-mapped IO **16**
- 6.1 The RAM . . . 16
- 6.2 The LED register and the UART . . . 16
- 6.3 The flash stall . . . 17

7 Boot modes and the SPI flash **19**
- 7.1 BRAM boot against flash boot . . . 19
- 7.2 The flash layout . . . 19
- 7.3 CCLK and STARTUPE2 . . . 20
- 7.4 Execute-in-place . . . 20

8 From C to the board **22**

9 Verification **24**
- 9.1 The ALU unit bench . . . 24
- 9.2 The golden trace . . . 24
- 9.3 The regression . . . 25
- 9.4 The gates . . . 25

10 Implementation results **26**
- 10.1 Utilization . . . 26
- 10.2 Timing . . . 26
- 10.3 Power . . . 27
- 10.4 The design on the device . . . 28

11 Results on the board **30**
11.1 Mandelbrot in BRAM boot 30
11.2 riscv_logo 30
11.3 tinyraytracer in flash boot 30
11.4 ST_NICCC 32
11.5 blinker and sieve 32

12 For the next student **33**
12.1 Pitfalls 33
12.2 Extensions, ranked 35

13 Conclusions **37**

A Memory map **38**

B Make targets **39**

C ISA coverage **41**

Bibliography **42**

1 Introduction

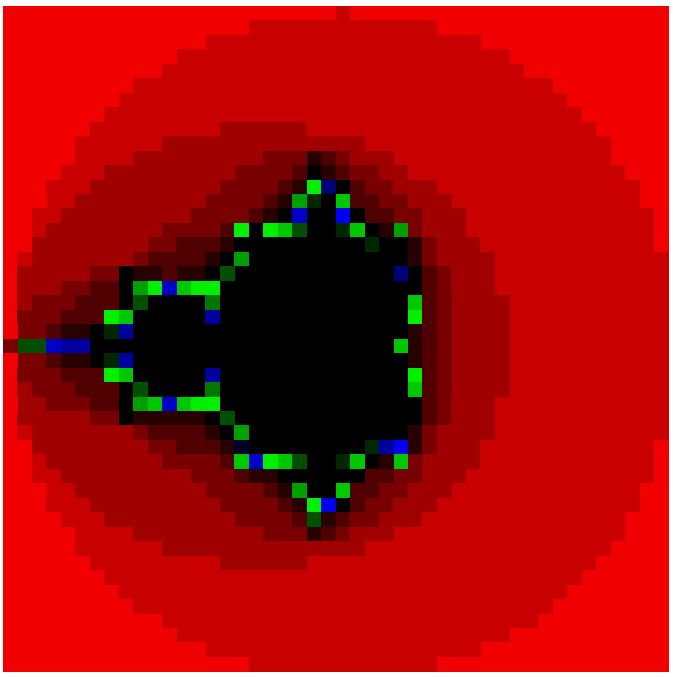


Figure 1.1: What the system does: the mandelbrot demo's first frame, rendered from the board's UART capture as a terminal shows it, 46 rows of 46 pixels, each pixel two spaces on a 24-bit background colour. The frame is byte for byte the one the simulation produced and the one the C source prints when compiled for a PC (Chapter 11).

1.1 The project

This is an undergraduate project with one aim: to build a processor that runs compiled C on a real FPGA board, and to understand every layer between the C source and the lit LED. The result is the system on chip this report describes: a multi-cycle RV32I core, a few kilobytes of RAM, a handful of memory-mapped IO registers and a flash controller, wired into one top level, synthesised for the Kintex-7 of a Digilent Genesys 2, and run on that board at the board's 200 MHz clock. Figure 1.1 is what it does when the bitstream loads: it draws the Mandelbrot set over the UART, one frame a second.

RISC-V was the natural instruction set. Its base integer set, RV32I, is 40 instructions, small enough for one student to implement and check instruction by instruction, and the GNU toolchain compiles plain C for it, so no compiler had to be written or bent. A multi-cycle core was the natural first design: one instruction at a time, one state of a state machine per clock cycle, with every memory wait visible as a state, so that the timing of the block RAM, the register file and the flash is honest in the state diagram rather than hidden in a pipeline. The Genesys 2 was the board at hand. Its device is two hundred times larger than the design needs, and the title page shows how much of the board waits for the next project.

The core and the SoC follow Bruno Levy's *learn-fpga* series *From Blinker to RISC-V* [1], from the multi-cycle core through memory-mapped IO and the GNU toolchain to execution from SPI flash. That series is the origin of the design's shape, its state machine and its bus, and of the demos; the port to the Genesys 2, the verification, the build flow, the board results and

this report are the project's own. The report does not account module by module for where the design diverges from its origin.

1.2 What was built

The hardware is eleven Verilog modules, one per file, in one clock domain (Chapter 4). The core sees a single bus, a 32-bit address with a read strobe, write data with a byte mask and a busy input, and `IOBus` decodes that bus into three regions by one address bit each: the RAM, the IO registers and the flash. The RAM is 6 KB of block RAM, shared by instruction fetch and data; the IO registers are the LEDs, the UART's data register and its status word; the flash controller turns a read in flash space into one SPI transaction and holds the core busy for its length. The clock enters through the board's differential input, and the reset is the board's button, synchronous everywhere.

The core (Chapter 5) executes RV32I and nothing beyond it: no multiplier, no interrupts, no control and status registers. Its state machine has seven states, and an instruction takes four cycles, a store five and a load six, each cycle accounted for by a memory or register-file read that is answered one cycle after it is asked. The same hardware boots two ways (Chapter 7). In BRAM boot the firmware image is read into the block RAM when the bitstream is synthesised, so a program is part of the bitstream. In flash boot the core's reset vector points into the board's flash, and it fetches each instruction from there in place, which allows programs far larger than the RAM at a hundredth of the speed.

The firmware (Chapter 8) is six demos in C and assembly, built by the RISC-V GCC with a start-up file and a linker script for each boot mode, and an ELF-to-hex converter that writes the firmware image. The repository (Chapter 3) holds the RTL, the firmware, the simulation, the Vivado scripts, this report and its figure scripts, and a set of make targets that build and check all of it; the Vivado project is generated, not committed. Verification (Chapter 9) is an ALU unit bench, a golden trace of the mandelbrot demo that the regression must reproduce byte for byte and cycle for cycle, and a lint; each is a gate whose log a commit cites.

1.3 Results in brief

- **Size and speed.** The routed design is 984 LUTs, 244 flip-flops and two block RAM tiles, under half a percent of the XC7K325T. The 200 MHz clock is met with 0.219 ns of slack on the worst path, which is the load path: the unregistered load-store address through the region decode into the register-file write. Vivado estimates 194 mW on the chip, 160 mW of it the device's static power (Chapter 10).
- **The board, the simulation and the source agree.** The first mandelbrot frame the board sent, 11 014 bytes, is byte for byte the UART output of the golden trace, and that is byte for byte what the C program prints when compiled natively. The board's frame time, 1031.8 ms, matches the simulation's 206 292 808 cycles at 200 MHz, 1031.46 ms, within the timestamps' accuracy (Chapter 11).
- **Flash boot works.** tinyraytracer, 29 KB of code and constants against 6 KB of RAM, rendered its picture from the flash in 127 s, with one press of the reset button after each load, which pitfall P5 explains. riscv_logo, sieve and the blinker ran in BRAM boot; ST_NICCC was not run.
- **The demos are bound by the UART.** At 115 200 baud a byte takes 87 µs, and the mandelbrot frame spends about 956 of its 1032 ms sending; the core's own work is the rest. A faster core would not show on these demos.

- **Ten pitfalls are recorded**, P1 to P10 in Chapter 12, each with its symptom, its cause and where the design or the repository now handles it. The costliest were outside the core: the board's UART net names, tools run from the wrong directory, a signed shift made unsigned by Verilog's rules, the flash clock's hand-over after configuration, and a flash write that clears a bit the factory bitstream needs.

1.4 How to read this report

Chapters 2 to 8 describe the system from the outside in: the board and the tools, the repository, the architecture, the core, the memory and the IO, the boot modes, and the path from a C file to the board. Chapters 9 to 11 are the evidence: how the design is checked, what Vivado made of it, and what the board did. Chapter 12 is for whoever extends the design, and Chapter 13 concludes. The appendices hold the memory map, the make targets and the ISA coverage.

A few terms are used in one sense throughout, in the report, the code and the repository's documents. The *multi-cycle core* is this CPU, as against a single-cycle or a pipelined one. *BRAM boot* and *flash boot* are the two boot modes. The *firmware image* is the one program loaded into the RAM at configuration, `rtl/firmware.hex`. A *demo* is one of the six programs under `fw/`. *Memory-mapped IO* is how the LEDs and the UART are reached, by ordinary loads and stores. The *final report* is this document. The design notes, `docs/design/notes.md`, hold the full reasoning behind each module; the report states each choice and cites the notes' section by its heading, as (`notes.md`, "section"). Every figure that a script can make is regenerated by `make figures` in `docs/design/`, and the recipe for each manual one is in the figures' README.

2 Board and tools

The Digilent Genesys 2 carries a Kintex-7 XC7K325T, speed grade −2, in the 900-ball FFG900 package, with 1 GB of DDR3, HDMI and DisplayPort, Ethernet, audio, an FMC connector and five Pmods around it [3]. The design touches seven of its parts, five of them through the nine ports of `SOC`, 16 pins, and two only to be configured (Table 2.1); everything else on the board is room to grow (Chapter 12). Figure 2.1 shows those five and the mode jumper in Digilent's schematic [4], and Figure 2.3 shows where their pins sit on the package.

Part (schematic page)	FPGA pins	Design port
200 MHz LVDS oscillator, DSC1103 (IC23, p. 13)	AD12, AD11: an MRCC pair of bank 33	`sysclk_p`, `sysclk_n`
CPU RESET button (BTN1, p. 11)	R19	`RESET`, low while pressed
LEDs LD0 to LD7 (p. 4)	T28, V19, U30, U29, V20, V26, W24, W23	`LEDS[0]` to `LEDS[7]`, high to light
USB-UART bridge, FT232RQ (IC16, p. 9), on the micro-USB port J15	Y23, Y20	`uart_rx_out`, `uart_tx_in`
QSPI flash, S25FL256S, 32 MB (IC26, p. 15)	U19, P24, R25, and CCLK on B10	`spiflash_csn`, `spiflash_mosi`, `spiflash_miso`, and the clock through `STARTUPE2`
USB-JTAG, on the micro-USB port J17	the JTAG pins	none: it loads bitstreams
Mode jumper JP5 (p. 15)	M2 to M0	none: it picks what configures the FPGA at power-on

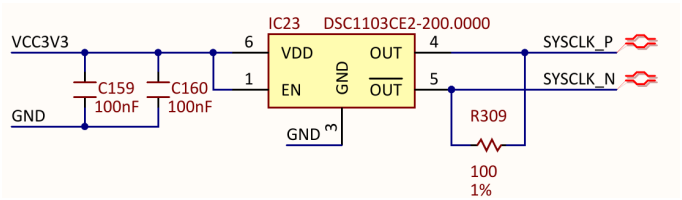
Table 2.1: The Genesys 2 parts the design uses. Pins and port names are those of `constraints/Genesys-2-Master.xdc`; parts and pages, those of Digilent's schematic, revision H.1 [4].

Clock, reset and LEDs. The oscillator drives a differential pair and nothing else, so the clock enters through `IBUFDS`, on a clock-capable pin pair (Section 4.3). The reset button grounds `CPU_RESET_N` against a 10 kΩ pull-up: the line idles high and goes low while the button is held, hence the active-low reset of the design. Each LED is driven through 330 Ω to ground, so a 1 in the LED register lights it.

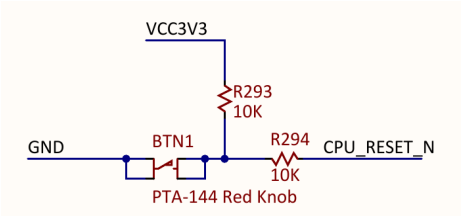
UART. The FT232R turns the USB port into a serial line. The board's net names take the PC's view for TX and RX and the FPGA's for in and out: `UART_TX_IN` is the PC's transmit line, buffered by IC25D, into the FPGA, and `UART_RX_OUT` is the FPGA's transmit line into the bridge's RXD. The design only transmits, on `uart_rx_out`; mixing the two up is pitfall P1 (`notes.md`, "UART port names and the pin-direction trap").

QSPI flash. The flash sits on the FPGA's configuration pins, so the FPGA can read its bitstream from it at power-on. The design uses it as plain SPI, one bit each way: `DQ0` carries the command and address, `DQ1` the data. Its clock is the dedicated CCLK pin, reached only through `STARTUPE2` (Section 7.3).

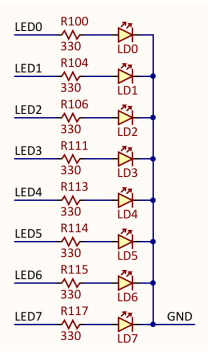
JTAG and the mode jumper. The second micro-USB port is Digilent's USB-JTAG circuit; Vivado's hardware manager loads every bitstream of this report through it, and writes the flash



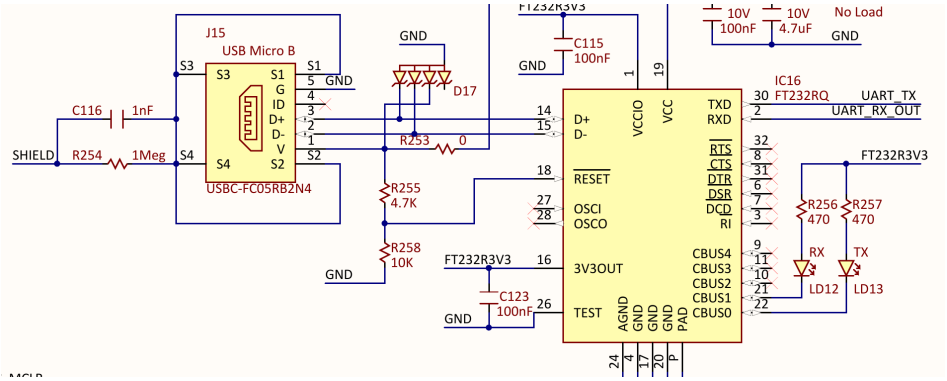
(a) The oscillator, IC23 (p. 13): an LVDS pair terminated by R309.



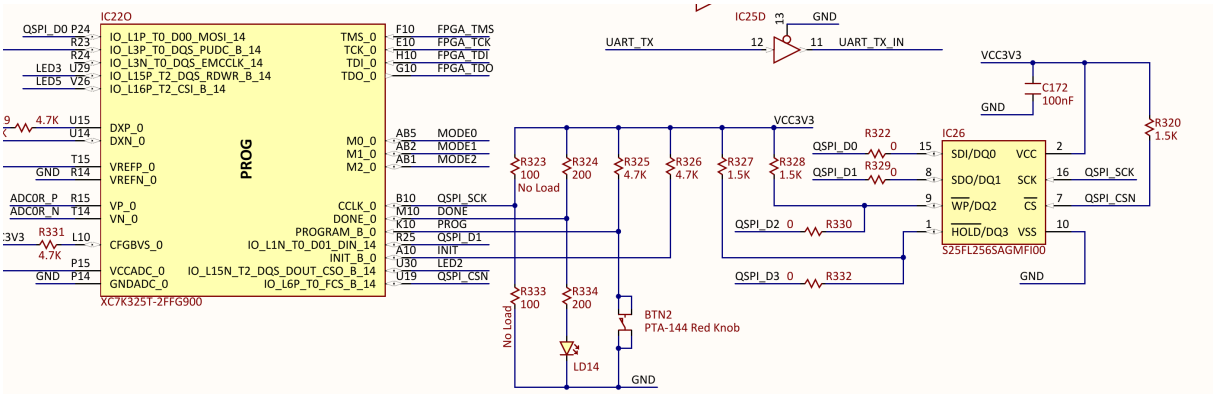
(b) CPU RESET, BTN1 (p. 11): pulled up, pressed to ground.



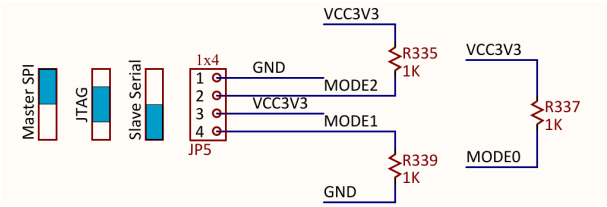
(c) LEDs, p. 4.



(d) The USB-UART bridge, IC16, behind the micro-USB port J15 (p. 9).



(e) The FPGA's configuration bank, PROG, and the QSPI flash, IC26 (p. 15). The flash's clock, `QSPI_SCK`, is the FPGA's CCLK pin, B10; at the top, the buffer IC25D carries the bridge's `UART_TX` to the FPGA as `UART_TX_IN`.



(f) The mode jumper, JP5 (p. 15), and its three positions.

Figure 2.1: The parts of Table 2.1 in Digilent's Genesys 2 schematic, revision H.1 [4], cropped at 300 dpi by `docs/design/scripts/schematic_crops.sh`.

through it (Chapter 7). JTAG works whatever the jumper says. The jumper, JP5, sets the mode pins read at power-on: Master SPI, $M[2{:}0] = 001$, configures the FPGA from the flash, which on the owner's board holds Digilent's factory demo at offset 0, left untouched; JTAG, 101, waits for a bitstream over JTAG; Slave Serial, 111, is not used here [5].

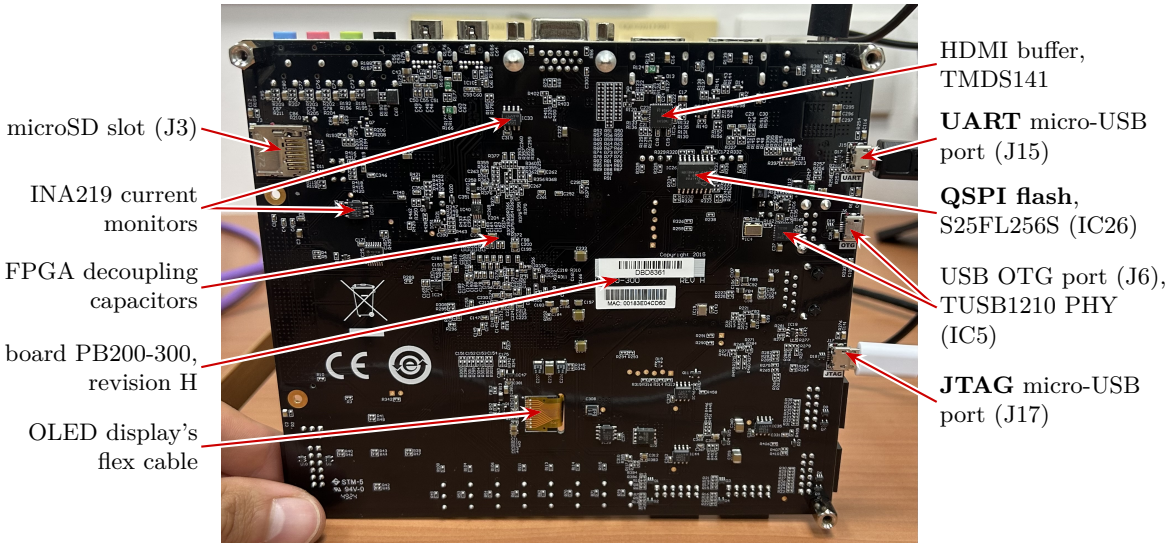


Figure 2.2: The underside of the Genesys 2, the top view mirrored: the UART and JTAG ports, on the left edge from above, are on the right here. Bold labels name the parts this design uses: the QSPI flash, which holds the bitstream, the demo data and the flash-boot program (Chapter 7), and the two micro-USB ports that carry its UART output and its programming. The parts are named from their markings and Digilent's schematic [4].

Colour	Module	Its ports
	the core: `Processor`, `Decoder`, `Control`, `ALU`	
	`RegisterFile`	
	`Memory`, the RAM	
	`IOBus`, the LED register	`LEDS`
	`UartTx`	`uart_rx_out`, `uart_tx_in`
	`SPIFlash`, with `STARTUPE2`	`spiflash_*`
	clock and reset	`sysclk_*`, `RESET`
	the worst setup path, selected	

Table 2.2: The module colours of every Vivado view in this report: Okabe-Ito colours, which stay distinct under the common forms of colour blindness. A selected path shows in Vivado's selection white.

2.1 Tools

Table 2.3 lists the tools the report and the repository were built with. Vivado numbers depend on the version: another release may place and route the same RTL differently, so the numbers of Chapter 10 hold for 2025.2.

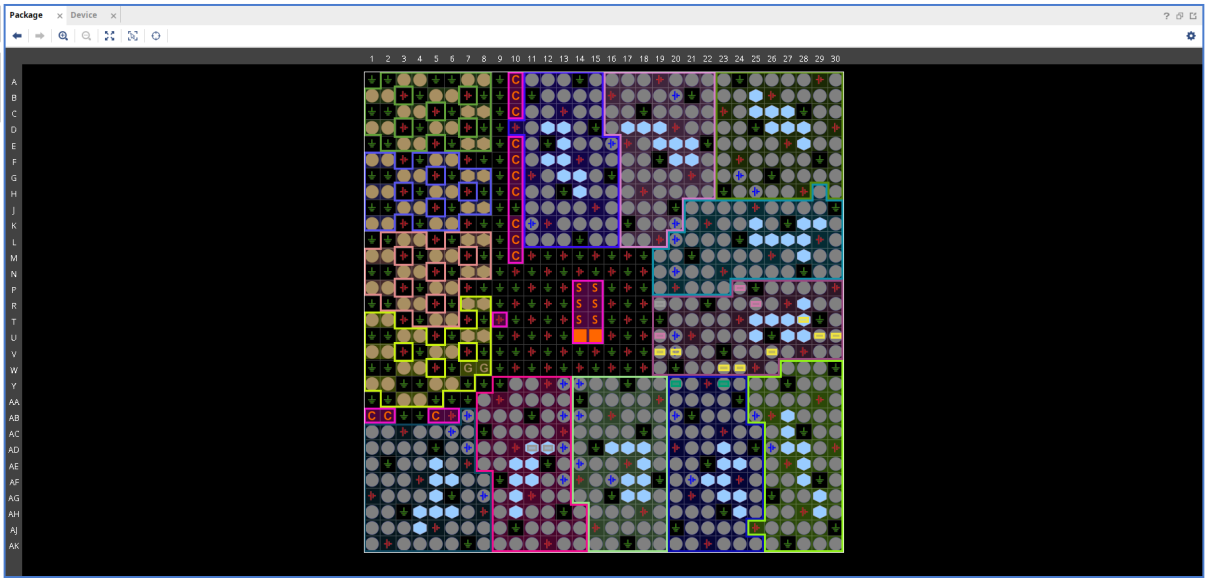


Figure 2.3: Vivado's Package view of the FFG900 package (Layout > I/O Planning), rows A to AK and columns 1 to 30, from the routed design. The design's 16 ports show in their modules' colours (Table 2.2): the eight LEDs, the three flash pins, the two UART pins, the reset button and the differential system clock. Fifteen of them are bonded IOBs; `uart_tx_in`, on Y20, has no load and so no IOB, but keeps its pin.

Tool	Version	Used for
AMD Vivado	2025.2	synthesis, implementation, the xsim regression, the hardware manager (JTAG and flash), the schematics and device views
`riscv64-unknown-elf-gcc`	13.2.0	the firmware, `-march=rv32i -mabi=ilp32` (Ubuntu's `gcc-riscv64-unknown-elf`, with binutils 2.42)
Icarus Verilog	12.0	the ALU unit bench and the waveforms
Python	3.12, Pillow 10.2, pyserial 3.5	figure scripts, board captures
Ghostscript, Poppler	10.02.1, 24.02	cropping figures
pdfTEX	TeX Live 2023	this report

Table 2.3: Tool versions, on Ubuntu 24.04.

3 Repository tour

The repository is a plain source tree. Its Vivado project is not committed but generated, by `vivado/create_project.tcl`, into `build/vivado/`, and refers to the sources where they are; what the tools generate, git ignores, nearly all of it under a `build/` directory. Table 3.1 gives the folders and Table 3.2 the make commands a change starts from; Appendix B lists every target, with what it needs.

Folder	What it holds
`rtl/`	The synthesisable Verilog, one module per file: `SOC.v`, the top; `Processor.v` with `Decoder.v`, `ALU.v`, `RegisterFile.v` and `Control.v`; `IOBus.v`, `Memory.v`, `UartTx.v`, `SPIFlash.v` and `Clockworks.v`; `defines.vh`, the memory map and the bus widths; `firmware.hex`, the one firmware image
`fw/`	The demos, in C and assembly, their start-up code and linker scripts, the `Makefile`, the ELF-to-hex converter and `write_mcs.tcl`; `tests/` holds the firmware check
`sim/`	The SoC testbench, `bench.v`, the golden trace in `golden/` and `run_regression.sh`; the ALU unit bench, `ALUBench.v`, and `run_unit.sh`; `asm/`, a Verilog-macro assembler for test programs
`constraints/`	`Genesys-2-Master.xdc`: the pins and the 200 MHz clock
`vivado/`	The Tcl scripts that regenerate the project, build the bitstream and run the non-project baseline, with `common.tcl` shared by all three; `rtl_sources.txt`, the file list every script reads
`docs/`	`CONTRIBUTING.md`, the coding conventions and the gates; `baseline.md`, the baseline trace and synthesis numbers; `design/`, this report's sources, its figures and their scripts, the board captures, the design notes, and `scripts/flashregs/`, a small FPGA design that reads and sets the flash's Quad Enable bit
`scripts/`	`lint.sh`, the pre-commit hook that runs it, the commit-msg hook that checks cited gate logs, and `quote_comment.sh`, which moves an RTL comment into the design notes
`build/`	Everything generated: the Vivado project, run directories, `SOC.bit`, the gates' logs. Not tracked

Table 3.1: The folders, as the README's "Repository layout" lists them.

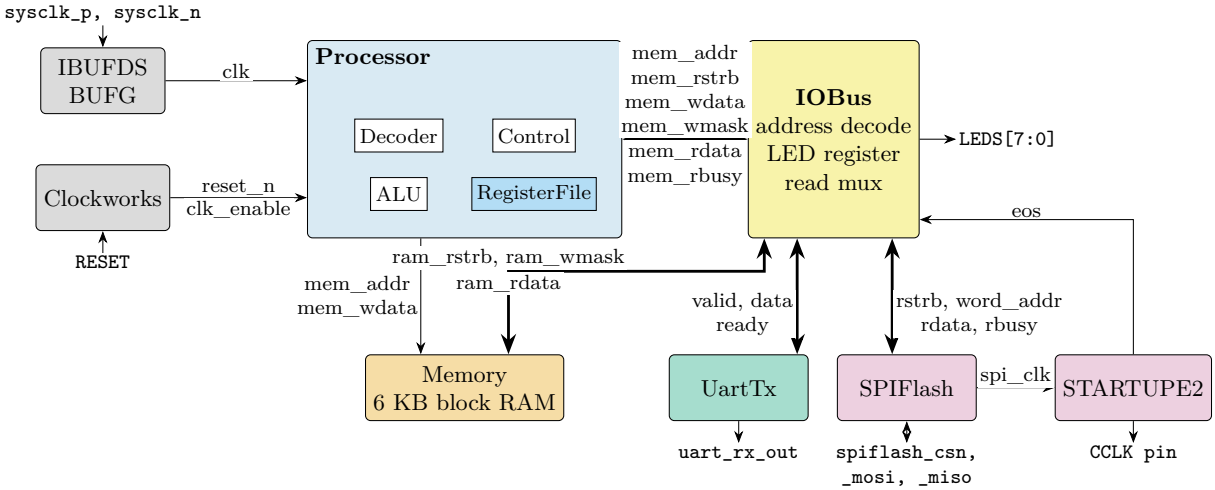
A first reading of the sources can follow the bus outwards: `rtl/defines.vh` for the memory map, `rtl/SOC.v` for the wiring, `rtl/Control.v` and `rtl/Processor.v` for the core, then `fw/start.S` and `fw/mandelbrot.c` for the program the regression runs. The design notes, `docs/design/notes.md`, are grouped by module, and each `notes.md` citation in this report names one of their sections.

Command	What it does
`make lint`	The tool-free checks of `scripts/lint.sh`, in seconds
`make unit`	The ALU unit bench, under iverilog, in seconds
`make smoke`	`unit`, the firmware check, then the regression's first 4 million cycles: about 25 s
`make -j2 check`	`unit`, then the firmware check, the full regression and Vivado synthesis: about 6 minutes. Its parts run alone as `check-fw`, `check-sim` and `check-synth`
`make golden`	Re-captures the golden trace, for an intended change of behaviour only
`make check-moves BASE=`*rev*	Every file moved since *rev* is a byte-identical rename
`make bitstream`	Synthesis, implementation and `build/SOC.bit`, in flash boot
`make baseline`	The non-project run behind `docs/baseline.md`, with its reports in `build/baseline/`: about a minute
`make -C fw`	Mandelbrot in BRAM boot, written to `rtl/firmware.hex`; `PROG=` picks another demo and `TARGET=flash` builds for flash boot
`make -C docs/design`	This report, from the committed figures; `figures` regenerates them

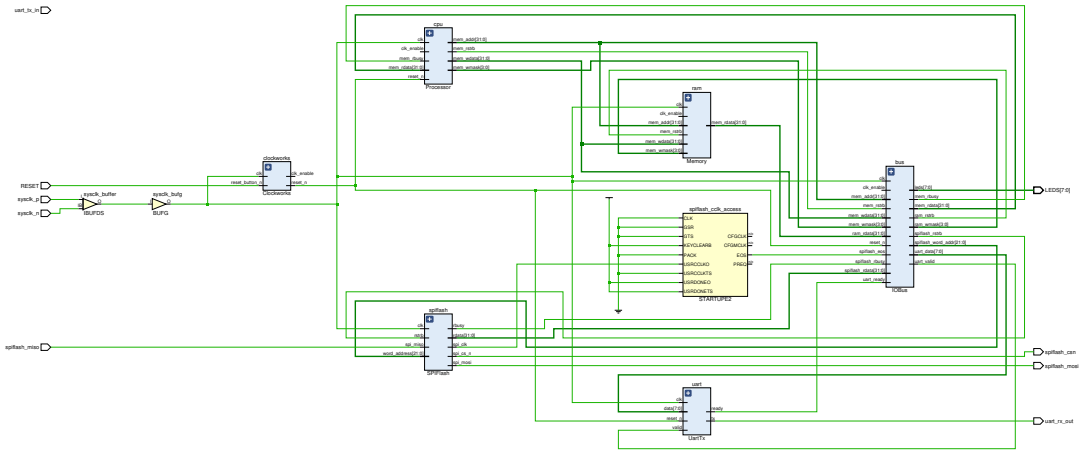
Table 3.2: The make entry points. The first eight are the targets of the root `Makefile`, whose header says what each runs; Chapter 9 says when each one runs.

4 System architecture

The SoC is six modules in one clock domain, wired together by `SOC`, the top level (Figure 4.1), with four more inside the core. The core sees one bus; everything else hangs off that bus. This chapter gives the bus and the modules, the memory map and its decode, and the clock and reset. Each section names the design-notes section that holds the full reasoning.



(a) The block diagram, from `rtl/SOC.v`, in the module colours of Table 2.2. The thick arrows are the core's bus and the handshakes `IOBus` derives from it. `clk`, 200 MHz, reaches every block; `reset_n` and `clk_enable`, drawn into the core only, also reach `IOBus`, the RAM (`clk_enable`) and the UART (`reset_n`).



(b) Vivado's RTL schematic of the same top level, elaborated from the Verilog with `synth_design -rtl`: the six instances under their names in `SOC.v`, and the three primitives, `IBUFDS` and `BUFG` on the clock and `STARTUPE2` on the flash clock.

Figure 4.1: The SoC as drawn and as Vivado inferred it.

4.1 The bus and the modules

The core, `Processor`, sees one bus and nothing else: a 32-bit byte address, a one-cycle read strobe, 32 bits of read data, 32 bits of write data with a four-bit byte mask, and a busy input. The bus has no acknowledge. A read requested with `mem_rstrb` is answered on `mem_rdata` in

the next cycle, unless `mem_rbusy` is high, and a write lands on the edge where `mem_wmask` is non-zero. Every address falls in some region, so no access is refused (`notes.md`, "Processor bus has no notion of RAM versus device").

Everything around the core follows from that contract. `IOBus` decodes the address into three regions, forwards the strobe and the mask to the RAM, holds the LED register, turns a store into the UART's send pulse and a flash read into an SPI transaction, and multiplexes the read data back. `Memory` is 6 KB of block RAM with one port, shared by instruction fetch and by loads and stores: there is no separate instruction memory. Its read is registered, which is why the core waits a cycle after every read, and its write lands on the same edge, which is why a store needs no wait (`notes.md`, "Read and write timing are not symmetric"). `UartTx` sends 8N1 at 115 200 baud and `SPIFlash` reads one 32-bit word per transaction, at a sixth of the system clock. The flash controller is the only block that stalls the core, by holding `mem_rbusy` for the length of a transaction; the RAM and the IO registers always answer in the next cycle (`notes.md`, "Only flash reads stall the core; read mux"). Chapters 6 and 7 give each module's timing. Figure 4.1b is what Vivado makes of `SOC.v` before synthesis: the same six instances. `Clockworks` is among them but contributes no cell to the netlist, as Section 4.3 explains.

4.2 Memory map and address decode

Table 4.1 is the memory map, defined once in `rtl/defines.vh`. The firmware carries copies of its values in `fw/io.h` and the start-up code, and a lint rule holds every copy equal to the defines (Chapter 9).

Byte address	Selected by	Region or register	Access
`0x0000_0000`–`0x0000_17FF`	no region bit	RAM, 6 KB (1536 words)	read, write, fetch
`0x0040_0000`	bit 22	`IO_BASE`, the IO registers	
`+ 0x04`	bit 2	`IO_LEDS`, the LED register	write
`+ 0x08`	bit 3	`IO_UART_DAT`, send one byte	write
`+ 0x10`	bit 4	`IO_UART_CNTL`, status; bit 9 = busy	read
`0x0100_0000`–`0x01FF_FFFF`	bit 24	SPI flash, a 16 MB window on the chip	read, fetch
`0x01D0_0000`–`0x01FF_FFFF`		the code window, 3 MB, where flash boot starts	

Table 4.1: The memory map (`rtl/defines.vh`). The reset vector is `0x0000_0000` in BRAM boot and `0x01D0_0000` in flash boot (Chapter 7).

A region is chosen by one address bit, not by a comparator: bit 24 selects the flash, bit 22 selects the IO registers when bit 24 is clear, and an address with neither bit set is RAM. Inside the IO region each register owns one bit as well, bits 2, 3 and 4, so a store to `IO_BASE + 4` lights the LEDs and a store to `IO_BASE + 8` sends a byte. The whole decode is a few bit tests in `IOBus`, the address masked with a base and OR-reduced, and a register is as cheap as a region (`notes.md`, "Address decode: one bit per region, one-hot IO registers, why bit 24").

The flash bit is the one that was chosen with care, because it fixes how much of the chip is reachable: everything below the selector bit is the offset inside the flash. Bit 23 would leave 8 MB, which does not reach past the 10.9 MB bitstream that sits at the start of the same chip; bit 24 leaves 24 bits, 16 MB, exactly the three address bytes of the flash's `03h` read command. The IO bit only has to lie between the RAM and the flash, and bit 22 does. The code window at 13 MB, and why, are in Chapter 7.

The decode has three consequences worth knowing before writing firmware. Nothing is rejected: an address with no region bit selects the RAM, past its 6 KB end as well, and what such an access does is undefined; in simulation it reads `x`. Only the UART status register answers

a read; the LED and UART data registers, and any other IO address, read as zero. And a write to flash space does nothing: the controller only reads. To add a peripheral, give it the next free bit of the IO word address, bit 5 at byte offset `0x20`, decode it in `IOBus` as the three existing ones are decoded, add it to the read mux if it answers reads, and add its copy in `fw/io.h` to the lint rule.

4.3 Clock and reset

The board's 200 MHz oscillator drives a differential pair, so the clock enters through `IBUFDS`, which turns the pair into one signal, and `BUFG`, which puts it on a global clock net (`notes.md`, "Genesys 2 differential clock input"). That net, `clk`, is the only clock in the design; its 5 ns period is the one timing constraint, met with 0.219 ns to spare (Chapter 10).

`Clockworks` derives two signals from the clock and the button. `clk_enable` is a gearbox: with the parameter `SLOW` set to n, it is high for one cycle in 2^n, and the core, the bus and the RAM advance only on those cycles, while the UART and the flash controller run every cycle, so the baud rate and the SPI clock stay right (`notes.md`, "Gearbox counter"). With `SLOW` at 20 the core runs a million times slower and the LEDs can be watched; the shipped design sets it to 0, `clk_enable` is the constant 1, and the module synthesises to nothing.

`reset_n` is the CPU RESET button, active-low on this board, forwarded as it is: no synchroniser and no debounce. Every reset in the design is synchronous, and three places have one: the core's `pc`, `instr` and FSM state, the LED register, and the UART's transmitter. The register file, the RAM and the flash controller have none; the register file because the core fetches an instruction before it reads any register. The core's and the LED register's resets are not gated by `clk_enable`, so a slowed design still resets on the next edge (`notes.md`, "Synchronous reset and the clock enable"). The core's three registers also declare their reset value as their initial value, which is what configuration loads into the flip-flop on a 7-series part, so the core leaves configuration already in its reset state and starts without a press of the button (`notes.md`, "Power-on state matches the reset state"). The LED register and the UART declare none and configure to zero: the LEDs dark, and the UART in its idle state, whose first clock raises the line and `ready`. In flash boot a press is still needed on the board, for a reason that lies in the flash controller's start-up, not here (pitfall P5).

The one subtlety of the reset is where the FSM lands: in `WAIT_DATA`, which asserts no strobe and falls through to `FETCH_INSTR` in the first cycle after the button is released, and not in `FETCH_INSTR` itself. Pitfall P6 gives the reason (`notes.md`, "Reset lands in WAIT_DATA, not FETCH_INSTR").

5 The multi-cycle core

`Processor` executes RV32I, the base integer instruction set, and nothing beyond it: no M extension, no interrupts, no control and status registers; `ECALL` and `EBREAK` stop the core (Appendix C). It is a multi-cycle core: one instruction at a time, several clock cycles each, one FSM state per cycle. `Control` is the FSM; `Decoder`, `ALU` and `RegisterFile` are what their names say; `Processor` holds `pc`, the instruction register and the glue between the other four.

5.1 The state machine

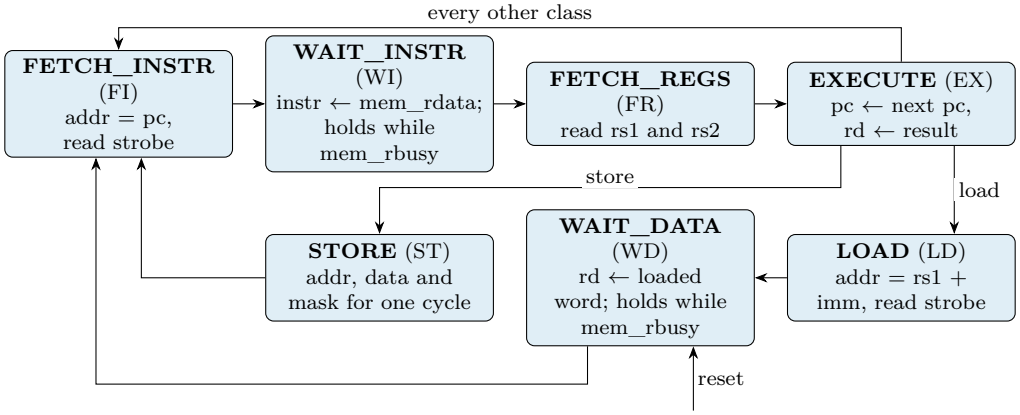


Figure 5.1: The control FSM of `rtl/Control.v`: seven states, one cycle each, except the two waits, which hold while the flash controller is busy. The two-letter codes are the ones Figure 5.3 uses.

An instruction takes several cycles because the RAM answers a read one cycle after it is requested, and because the register file's read is synchronous too, so each of those needs its own state (`notes.md`, "Why one instruction takes several cycles"). Figure 5.1 shows the seven. `FETCH_INSTR` drives `pc` on the bus with the read strobe; `WAIT_INSTR` latches the word the RAM returns; `FETCH_REGS` latches `rs1` and `rs2`; `EXECUTE` loads `pc` with its next value and writes `rd` for every instruction but a load, a store or a branch. A load goes on to `LOAD`, which drives the data address with the strobe, and `WAIT_DATA`, which writes the returned word to `rd`. A store goes to `STORE`, which drives the address, the data and the byte mask for one cycle; the RAM writes on that same edge, so there is no wait after a store (`notes.md`, "STORE needs no wait state"). The two waits are the only states that can last longer than a cycle: they hold while `mem_rbusy` is high, which only the flash controller raises (`notes.md`, "Instruction fetch stalls on mem_rbusy").

`Control` decodes four one-cycle strobes from the state, `latch_instr`, `read_regs`, `update_pc` and `write_rd`, already qualified with `clk_enable` and `reset_n`, so each datapath register tests only its strobe. `write_rd` in `WAIT_DATA` is gated by `mem_rbusy` as well, or a stalled flash load would write the bus's garbage to `rd` on every stalled cycle (`notes.md`, "Write-back enable gated by mem_rbusy in WAIT_DATA"). Table 5.1 gives the cost per instruction class.

Two of the four cycles of the commonest class are the fetch. A pipelined core would overlap the fetch of one instruction with the execution of the last and retire close to one instruction per cycle: a gain of four to six on code bound by computation. The demos of Chapter 11 are bound by the UART, which sets their pace at any clock, so for them the multi-cycle core costs nothing visible. Pipelining is the first extension in Chapter 12.

Instruction class	States	Cycles
ALU, register and immediate; LUI, AUIPC; JAL, JALR; branch	FI WI FR EX	4
Load: LB, LH, LW, LBU, LHU	FI WI FR EX LD WD	6
Store: SB, SH, SW	FI WI FR EX ST	5
SYSTEM: ECALL, EBREAK	FI WI FR EX, `pc` unchanged	4, repeated

Table 5.1: Cycles per instruction class from the RAM. A SYSTEM instruction leaves `pc` alone, so the core fetches it again, forever. Every instruction fetched from the flash, and every load from it, adds the SPI transaction to its wait state (Chapter 6).

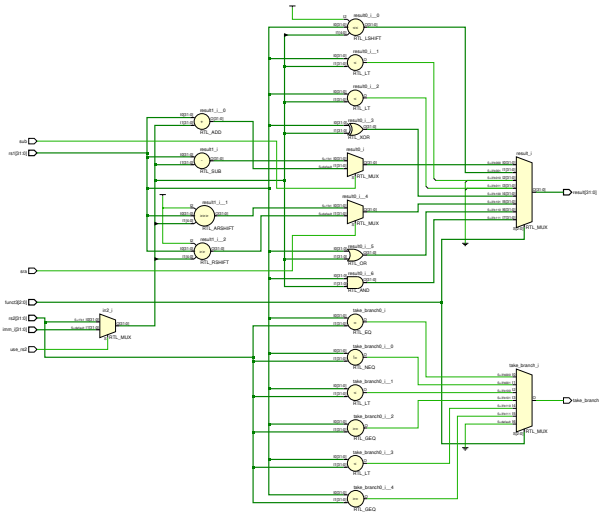


Figure 5.2: Vivado's RTL schematic of `rtl/ALU.v`, elaborated as its own top so that its ports frame the drawing. Upper half: the operand multiplexer on the left, then the adder and subtractor, the shifters, the comparators and the logic operations, into the result multiplexer on the right. Lower half: the branch comparators, into the multiplexer that yields `take_branch`. After synthesis nearly all of this merges into the LUTs around the register file (Chapter 10).

5.2 The datapath

`Decoder` is purely combinational on the instruction register: the ten class flags from the opcode, disjoint, so at most one is high and exactly one for every instruction the core implements; the register indices and the `funct3` and `funct7` fields, at fixed positions in every format; and the five immediates, sign-extended from bit 31. It also settles the two meanings of `funct7[5]`: a subtraction only in the register form, since there is no `SUBI` and an `ADDI` whose immediate happens to set bit 30 must still add, and an arithmetic shift in both forms (`notes.md`, "ADD versus SUB: funct7[5] alone is not enough").

`RegisterFile` is the 32 integer registers, in LUT RAM: a synchronous read of `rs1` and `rs2` on `read_regs`, a synchronous write of `rd` on `write_rd`, and `x0` never written and read as zero explicitly (`notes.md`, "x0 reads as zero explicitly"). `ALU` is combinational (Figure 5.2): its second operand is `rs2` for register instructions and the I-immediate for everything else, the shift amount is the low five bits of that operand either way, `funct3` picks one of the eight operations, and a second case on `funct3` computes the six branch conditions on `rs1` and `rs2`, with a default of zero so that no latch is inferred (`notes.md`, "Branch comparator default arm"). Both arms of the right shift are signed, or `SRA` shifts logically (pitfall P4).

The rest of `Processor` is the glue between these. A multiplexer chooses the next `pc`: the

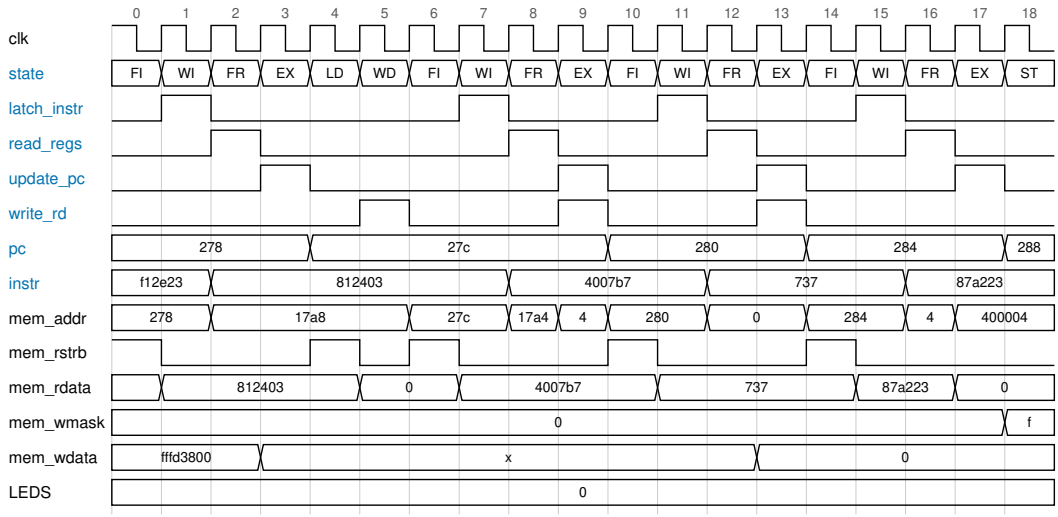


Figure 5.3: Four instructions of the mandelbrot demo in the simulated SoC, from the VCD of `docs/design/scripts/waves/WaveBench.v`: `lw s0,8(sp)` at `0x278`, `lui a5,0x400`, `lui a4,0x0` and `sw s0,4(a5)`, the store to the LED register. The core's signals are in blue, the bus in black; bus values are in hex with leading zeros dropped, and a value is left blank where it does not fit its cycle. `mem_wdata` reads `x` while the instruction's `rs2` field names a register no instruction has written yet: the register file has no reset.

branch target when a branch is taken, the jump target for `JAL`, `rs1` plus the immediate with bit 0 cleared for `JALR`, otherwise `pc + 4` (`notes.md`, "Why jalr_target is a named wire"). An adder forms the load or store address from `rs1` and the I- or S-immediate. The memory moves whole words, so a byte or halfword load picks its lane by the address's low two bits and sign-extends unless `funct3[2]` says not to, and a store broadcasts `rs2` into every lane it could land in and lets the byte mask select (`notes.md`, "Store data is broadcast to every lane; the mask selects"). The write-back multiplexer gives `rd` its value: `pc + 4` for the jumps, the U-immediate for `LUI`, `pc` plus it for `AUIPC`, the aligned load data, or the ALU result. The worst timing path of the design runs through this glue, from the instruction register through the load-store address adder, the region decode and the load lanes to the register file's write port (Chapter 10, pitfall P7).

5.3 One instruction through the states

Figure 5.3 is the simulated core at the end of `main`'s prologue in the mandelbrot demo, where the frame counter, just stored to the stack, is loaded back and written to the LEDs: four instructions, 19 cycles, every state of Figure 5.1.

The load takes cycles 0 to 5. In `FETCH_INSTR` the bus address is `pc`, `0x278`, with the strobe; in `WAIT_INSTR` the RAM answers with the instruction word, `0x00812403`, and `latch_instr` moves it into `instr` on the edge that ends the cycle. From then on the bus address is the load-store address, `sp + 8`. `EXECUTE` advances `pc` to `0x27c`; `LOAD` asserts the strobe at the data address; `WAIT_DATA` sees the word, 0, and `write_rd` writes it to `s0`. The two `LUI`s take four cycles each, cycles 6 to 13, writing `rd` in `EXECUTE`. The store, cycles 14 to 18, writes no register, so `write_rd` stays low in its `EXECUTE`; in `STORE` the bus carries the LED register's address, `0x400004`, the byte mask `f` and the data, and the register takes the value on the edge that ends the window. Each read is answered one cycle after its strobe, and no cycle is spent on the store: the two facts about the memory that shaped the FSM.

6 Memory and memory-mapped IO

The core reaches everything through the bus of Chapter 4, and to the core the three regions differ only in timing. The RAM and the IO registers answer every read in the next cycle and take every write on the edge it is presented; the flash answers a read about 380 cycles later, and the core waits. This chapter gives each target's side of the bus, in the order of the memory map (Table 4.1).

6.1 The RAM

`Memory` is 1536 words, 6 KB, of block RAM: two of the Kintex-7's 445 block RAM tiles. Its one port serves instruction fetch, loads and stores alike, so in BRAM boot the program, its data and its stack share it, the stack growing down from the top. The read is registered: the word addressed in the cycle of `ram_rstrb` appears on `ram_rdata` at the next edge, which is the one cycle every `WAIT` state of the FSM spends. A write needs no strobe and no wait: each of the four `ram_wmask` bits writes its byte lane on the edge where it is high, so `SB` and `SH` change one byte or halfword and leave the rest of the word as it was (`notes.md`, "Read and write timing are not symmetric") (`notes.md`, "Per-byte write enables"). `IOBus` masks both strobes off outside RAM space, so a store to an IO register never also writes the RAM word with the same index (`notes.md`, "RAM strobes masked outside RAM space").

The RAM's contents at configuration are the firmware image, `rtl/firmware.hex`, read with `$readmemh` when the bitstream is synthesised, so a BRAM-boot program is part of the bitstream and a new program is a new bitstream (Chapter 7). The file is read by bare name, relative to the working directory of the tool (`notes.md`, "Bare file name in $readmemh"), and a missing file leaves the RAM undefined with no error, as the header of `rtl/Memory.v` warns: pitfall P2.

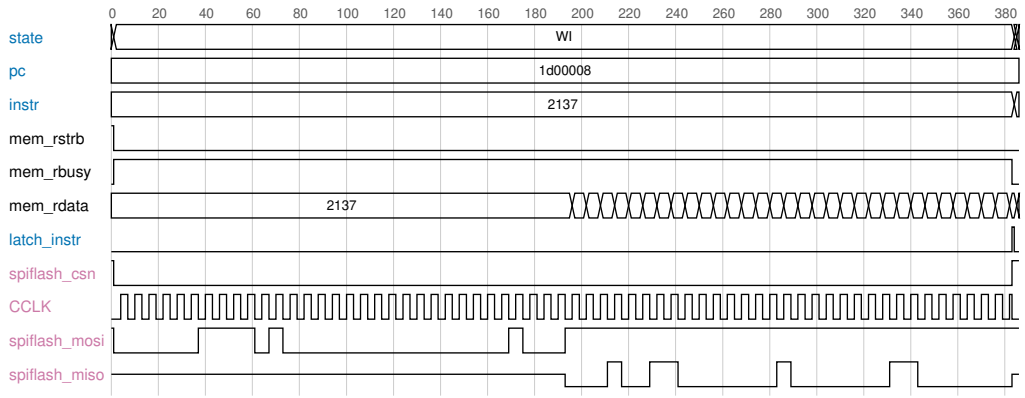
6.2 The LED register and the UART

The IO registers are three addresses in the IO region, one address bit each, and none of them is memory: a store to one is an action and a load from one is a question.

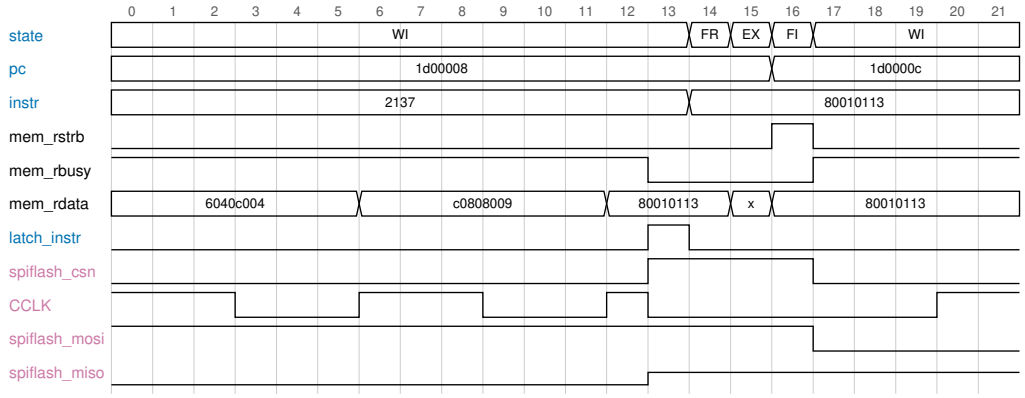
LEDs. A store to `IO_BASE + 4` sets the LED register to the low byte of the store data; `SW` and `SB` both work, since lane 0 carries the low byte of `rs2` in every store and `IOBus` ignores the byte mask. The register drives `LD0` to `LD7` directly, clears on reset, and reads as zero (`notes.md`, "LED register: reset not gated by the clock enable").

UART data. A store to `IO_BASE + 8` turns into `uart_valid`, a one-cycle pulse, with the low byte as `uart_data`. `UartTx` drops `ready` on the same edge and sends the byte as one start bit, eight data bits LSB first and one stop bit, each 1736 cycles long: 115 207 baud, 0.006 % above 115 200. A byte takes 17 360 cycles, 86.8 µs. The transmitter has no FIFO and ignores `valid` while it sends, so a byte stored too early is lost without a trace (`notes.md`, "UART send pulse and the status word (bit 9 convention)").

UART status. A load from `IO_BASE + 0x10` returns a word that is zero but for bit 9, which is `!ready`: the transmitter is busy. The word is not stored anywhere; `IOBus` computes it from the address and `ready` in the cycle the core reads it. It is the only IO address that answers a read. Bit 9 is a convention between `IOBus` and the firmware, held equal by the memory-map lint rule. `fw/putchar.S` is the whole driver: it stores the byte, then loads the status word until



(a) The whole instruction, 386 cycles: `FETCH_INSTR` at cycle 0, `WAIT_INSTR` for 383 cycles, then `FETCH_REGS` and `EXECUTE`.



(b) The end of the same transaction, 22 cycles from the last two data bits: chip select rises, `mem_rbusy` falls, the word is latched, and the next fetch starts the next transaction.

Figure 6.1: An instruction fetched from the flash, in the simulated SoC in flash boot (`docs/design/scripts/waves/WaveBench.v` with a model of the flash's read command, booting mandelbrot's flash build). The core's signals are in blue, the flash controller's pins in purple; `CCLK` is the flash's clock pin as Vivado's model of `STARTUPE2` drives it. While the word shifts in, `mem_rdata` shows the controller's shift register, byte-swapped. In `EXECUTE` the bus address is the instruction's load-store address, in RAM space, and `mem_rdata` reads the RAM's output register, not yet loaded: `x`. `spiflash_miso` is drawn mid-way where the flash leaves it undriven.

bit 9 is clear, so the next character never finds the transmitter busy. This wait is why the demos of Chapter 11 run at the pace of the UART.

6.3 The flash stall

A read in flash space is the one access the core waits for. In the cycle of the read strobe, `IOBus` passes it to `SPIFlash` with the word address, the low 22 bits of the word index; the controller drops chip select and raises `mem_rbusy` at the next edge, and holds both for the whole SPI transaction. The FSM's two wait states hold while `mem_rbusy` is high, and `write_rd` in `WAIT_DATA` is gated by it, so a stalled load writes nothing until its word has arrived (`notes.md`, "Only flash reads stall the core; read mux").

Figure 6.1 is one such stall, simulated: the third instruction of flash-boot start-up, fetched from `0x1D00008`. The transaction is 64 SPI clocks at a sixth of the system clock, 33.3 MHz: the opcode `03h`, the 24-bit byte address `0xD00008`, both on `spiflash_mosi`, then 32 data bits on `spiflash_miso`. That is 382 cycles of `mem_rbusy`, and the instruction takes 386 cycles, 1.93 µs,

where one fetched from the RAM takes 4. The word arrives lowest address first, each byte MSB first, as `13 01 01 80`; the controller swaps the bytes into `0x80010113`, `addi sp,sp,-2048`, little-endian as the RAM returns it (`notes.md`, "Byte order of the received word"). Chip select rises right after the last bit, and `mem_rbusy` with it; in that cycle `WAIT_INSTR` sees the word and latches it, and the core moves on as after any fetch. The controller's clock is held low while chip select is high, as SPI mode 0 needs, and its divider is held too, so the first clock pulse of a transaction is as long as the others (`notes.md`, "Divider held in reset while idle (short first pulse)") (`notes.md`, "CLK forced low while idle").

There is no cache and no prefetch: every word read from the flash is a transaction of its own, an instruction fetch as much as a load. Code that runs from the flash therefore runs about a hundred times slower than from the RAM, which is what shapes flash boot in the next chapter.

7 Boot modes and the SPI flash

The core starts at its reset vector, the `SOC` parameter `BOOT_ADDR`, and that one parameter is the boot mode. At `BOOT_ADDR_BRAM`, address 0, the core runs the firmware image held in the RAM; at `BOOT_ADDR_FLASH`, `0x01D0_0000`, it runs a program stored in the board's flash, in place. The hardware is otherwise the same (`notes.md`, "Why the reset vector is a parameter, and why a jump at address 0 is not enough").

7.1 BRAM boot against flash boot

	BRAM boot	Flash boot
Reset vector	`0x0000_0000`	`0x01D0_0000`, the default
Program held in	the RAM, from `rtl/firmware.hex`	the flash's code window
Room for code	6 KB, shared with data and stack	3 MB; data and stack in the RAM
New program	a new bitstream	a flash write; the bitstream stays
Start-up code	`fw/start.S`: `gp`, `sp`, `main`	`fw/start_spiflash.S`: also zeroes `.bss` and copies to the RAM
Speed	4 to 6 cycles an instruction	382 cycles more for each word read from the flash
Simulated	yes: the golden trace	the start-up only (Figure 6.1)

Table 7.1: The two boot modes. `make -C fw` builds either (`TARGET=bram` or `TARGET=flash`); the README's "Running the demos" gives the steps on the board.

Table 7.1 compares the two. BRAM boot is the simple one, and the one every simulation of the regression runs: the program is in the bitstream, starts at full speed, and fits in 6 KB, which mandelbrot, riscv_logo, sieve and the blinker do. Flash boot exists for the programs that do not fit: tinyraytracer is 29 KB of code and constants. The bitstream is built once, with the default `BOOT_ADDR`, and any program written to the code window runs on it. `make bitstream` builds that default; the BRAM-boot bitstreams of Chapter 11 override the parameter at synthesis, as the testbench does.

7.2 The flash layout

The flash is a 32 MB S25FL256S, and it also holds the bitstream the FPGA configures from at power-on, so a program has to fit around what is already there. Figure 7.1 is the layout `rtl/defines.vh` and `fw/memory.ld` assume (`notes.md`, "Flash memory map and boot address").

Every boundary is forced. An uncompressed 7-series bitstream holds every configuration frame whatever the design, so it is 11 443 728 bytes, `0xAE9E10`, for any design on this part, and it starts at offset 0, where the FPGA looks at power-on. The flash erases in 64 KB sectors, so a region written separately must start on a sector boundary: the scene data at 12 MB and the code at 13 MB leave a gap after each. The window ends at 16 MB because the controller sends the 24-bit address of the plain `03h` read, which is also why bit 24 selects flash space (Section 4.2); the upper 16 MB of the chip would need a four-byte address. Flash boot writes only the code window, with "Configuration File Only", and never offset 0, and the design is loaded over JTAG: the board keeps booting the bitstream it booted before, provided the write keeps the flash's Quad Enable bit (pitfall P8).

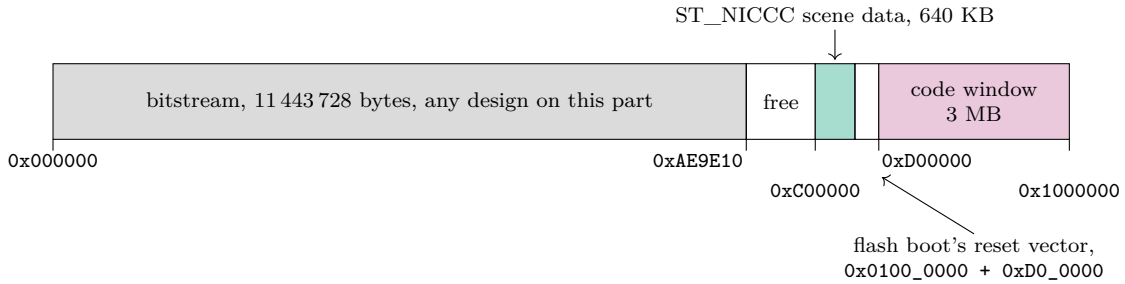


Figure 7.1: The flash layout of `rtl/defines.vh` and `fw/memory.ld`, to scale: the 16 MB the core reaches, at `0x0100_0000` in its address space, the lower half of the 32 MB chip. The bitstream from offset 0, ST_NICCC's 640 KB of scene data at `0xC00000`, and the code window at `0xD00000`, which `SPIFLASH_CODE_OFFSET` names. On the owner's board offset 0 holds Digilent's factory demo, whose data reaches up to `0xDEFFFF`; a flash-boot write replaces the part of it that the program covers (Chapter 11).

7.3 CCLK and STARTUPE2

Three of the flash's four pins, chip select and the two data lines, are multi-function pins of the FPGA: the configuration logic uses them at power-on and releases them to the design afterwards, so `spiflash_csn`, `spiflash_mosi` and `spiflash_miso` are ordinary ports in the constraints file. The clock is different. CCLK is a dedicated configuration pin, and a design can reach it only through the `STARTUPE2` primitive [5]: whatever drives its `USRCCLKO` input reaches the pin once configuration has ended. `SOC` wires `SPIFlash`'s clock to `USRCCLKO`, holds `USRCCLKTS` low so the pin stays driven, and takes `EOS`, end of startup, back to `IOBus`, which passes a flash read to the controller only once `EOS` is high (`notes.md`, "SPI flash CCLK access through STARTUPE2") (`notes.md`, "Flash read strobe gated by end-of-startup; 22-bit word address").

The hand-over is not immediate. Vivado's simulation model of `STARTUPE2` keeps CCLK high after end of startup until `USRCCLKO` has risen twice, so the first transaction after configuration loses two of its 64 clock edges and the flash misreads its command. In the simulation behind Figure 6.1 that alone costs the first fetch of `fw/start_spiflash.S`, with the `EOS` gate playing no part, and it is the most likely reason flash boot needs one press of the reset button after each load on the board. Pitfall P5 gives the mechanism and a candidate fix.

7.4 Execute-in-place

In flash boot the code is not copied into the RAM: the core fetches each instruction from the flash, one transaction per word, and `.text` and `.rodata` stay there (`notes.md`, "Instruction fetch stalls on mem_rbusy"). What must be writable cannot stay in the flash, so `fw/spiflash.ld` places `.data` and `.bss` in the RAM, with the initial values of `.data` stored in the flash right after the code, and `fw/start_spiflash.S` prepares the RAM before `main`: it sets `gp` to `IO_BASE` and `sp` to the top of the RAM, zeroes `.bss`, and copies the stored values into place, one flash read and one store per word.

The same copy moves the hottest code into the RAM. Every `*`, `/` and `%` in C is a call into `libgcc` on RV32I, and every character goes through `putchar` and the print routines, so the linker script puts `libgcc`'s multiply and divide, `fw/putchar.S`, `fw/print.c` and any function marked `.fastcode` in the copied section. The linker gives them RAM addresses, so the code in the flash calls them there. For mandelbrot's flash build that is 1 228 bytes copied, against 1 172 bytes of code and constants left in the flash. The copy costs time at start-up, six flash transactions per word, the load and the five instructions of the loop, which are fetched from the flash too: a few milliseconds for mandelbrot. It also costs RAM for the rest of the run: what

is copied shares the 6 KB with the data and the stack, and nothing checks that the stack stays clear of it.

8 From C to the board

Figure 8.1 follows one line of the mandelbrot demo, `IO_OUT(IO_LEDS,frame);`, which writes the frame counter to the LEDs, from the C source to the board, and the first instruction of mandelbrot's flash build from its ELF to the flash. The four instructions it becomes are those of Figure 5.3. Every excerpt is cut from the real build by `make flow`, part of `make figures`, which builds the firmware and a BRAM-boot bitstream in a scratch directory and fails unless each stage holds what the one before made: the image is the committed `rtl/firmware.hex`, its words are the ELF's instructions, the block RAMs hold the image, and the MCS holds the binary at the code offset.

From C to machine code. `make -C fw` compiles each C file with `gcc -c` and links the objects, the start-up code and `libgcc` with `ld` into an ELF (`notes.md`, "The compiler and linker flags, one by one"). The ELF holds machine code: the store is the 32-bit word `0x0087a223` at address `0x284`. The assembly in the figure is `gcc -S`'s output, written for the reader by `make -C fw build/mandelbrot.s`. `gcc -c` produces the same text as a temporary file and assembles it within the same command, so the build never keeps it, and nothing after the compiler reads it: there is no assembler step to look for, and the FPGA never sees an instruction's name. The assembly shows two things the linked code no longer does. `li a5,4194304` is a pseudo-instruction, one `lui` for `IO_BASE`, `0x400000`; and `%hi(.LC1)` names a string the linker has not placed yet, so the `lui` reads `0x0` only once the string lands below `0x800`, the first 2 KB.

The firmware image and the block RAMs. The firmware image is the ELF's bytes as text for `Memory.v`'s `$readmemh`: `firmware_words` writes four words a line, each most significant digit first, so the store is word 161, on line 41 (`notes.md`, "Why the ELF-to-hex conversion exists; endianness"). Synthesis turns the RAM into two `RAMB36E1`, each 2K deep and 18 bits wide: `ram_reg_0` holds the low 16 bits of every word and `ram_reg_1` the high 16, so the store's word is split between them, `A223` and `0087`, in `INIT_0A`, the `INIT` string of words 160 to 175. These `INIT` values become part of the bitstream, which is why a new program in BRAM boot needs a new bitstream (`notes.md`, "Simulation and hardware after a firmware change").

The bitstream, JTAG and the board. `SOC.bit` begins with a short header, the design's name, the tool version and the part, and then 11 443 612 bytes of configuration data, 11 443 728 bytes in all, the size of every bitstream for this part (Figure 7.1); the FPGA starts reading at the sync word, `0xAA995566` [5]. `board_program.tcl` loads it over JTAG in 7.2 s, from the script's announcement to Vivado's exit. The first byte of frame 0 arrives before Vivado has exited: the FPGA runs the design as soon as configuration ends, while the script still checks DONE. Frame 0 opens with the cursor-home sequence and the colour of its first pixel, and takes 1031.8 ms, which Chapter 11 compares with the simulation.

Flash boot. The same C source, linked with `fw/spiflash.ld`, runs from the flash's code window: its first instruction, `lui gp,0x400`, sits at `0x01D0_0000`. `objcopy -O binary` keeps only the bytes, in memory order, so the word `0x004001b7` becomes `b7 01 40 00`, little-endian; the firmware image writes the same kind of word the other way round, for `$readmemh`. `write_cfgmem` wraps the binary in an MCS file, Intel hex for the flash: a record giving the upper 16 bits of the address, `00D0`, then 16 data bytes a record. `board_flash.tcl` writes those bytes over JTAG and nothing else, the "Configuration File Only" range; the flash-boot bitstream is loaded as on the left, and the core fetches each instruction from the flash (Chapter 7). On the board, flash boot ran tinyraytracer, which does not fit in the RAM (Chapter 11); mandelbrot's flash build is shown as the same program.

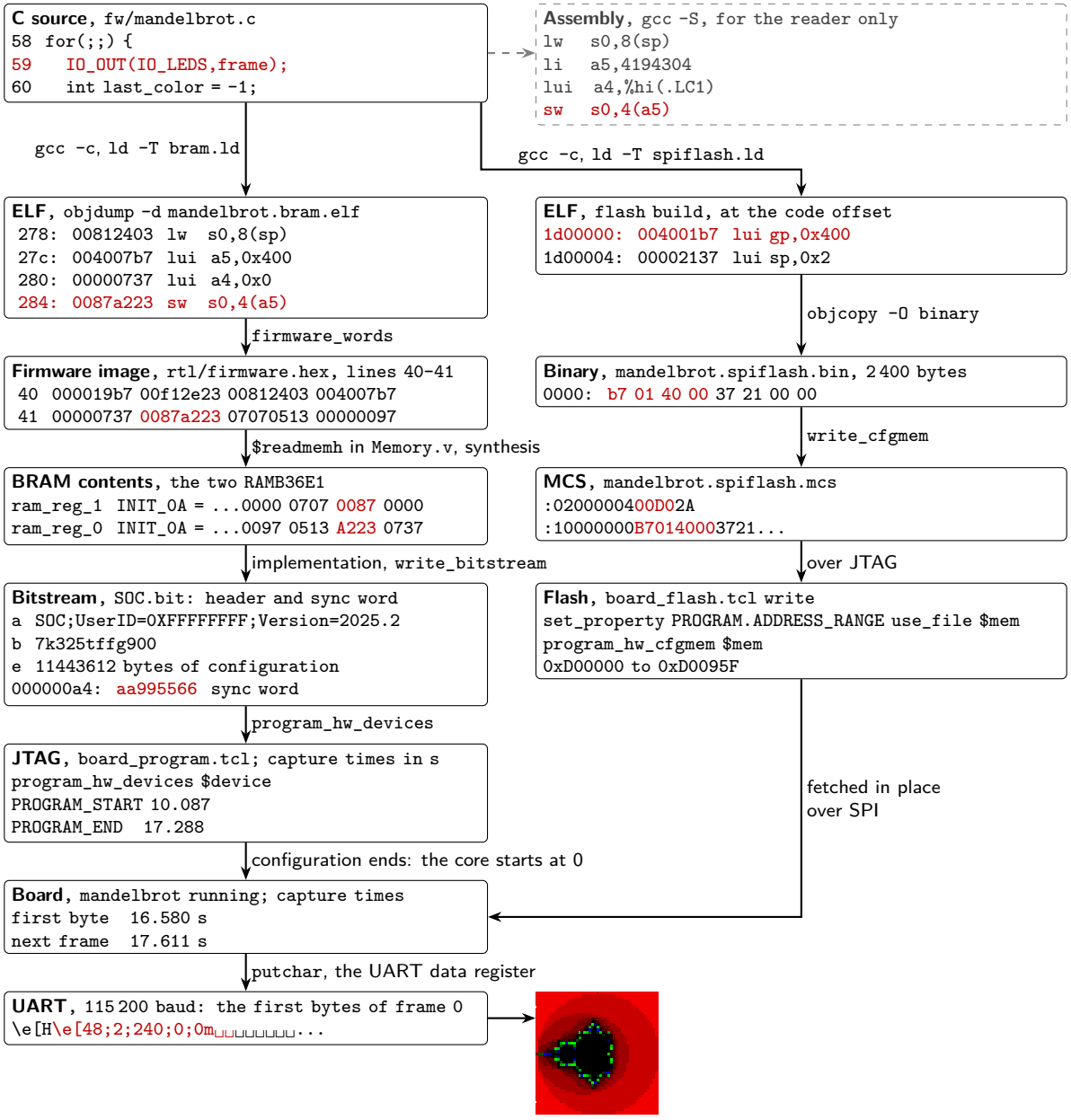


Figure 8.1: The mandelbrot demo from C to the board, in BRAM boot (left) and flash boot (right), with an excerpt of the real build at each stage, generated by `make flow`. In red, what the figure follows: line 59 and the store to the LED register it becomes, at `0x284`, in BRAM boot; the flash build's first instruction, in flash boot. The assembly, dashed, is made for the reader only, and no stage reads it. The BRAM halves are spaced 16 bits apart, the most significant first, as the `INIT` string writes them; the UART bytes write `\e` for ESC and ␣ for a space, and the frame they open is drawn beside them (Figure 1.1). In flash boot the bitstream comes over JTAG as on the left; the board's times are those of BRAM boot, the only way mandelbrot ran on the board.

9 Verification

The design is checked at three levels: one module against values worked out by hand, the whole SoC against a recorded trace, and the repository against mechanical rules. A make target runs each check, and each target, a gate, leaves a log that a commit cites.

9.1 The ALU unit bench

`sim/ALUBench.v` drives the ports of `rtl/ALU.v` directly and compares ten right shifts with results worked out by hand from the RV32I specification [2], not from the RTL: `SRL`, `SRA`, `SRLI` and `SRAI` on negative, positive and most-negative operands, by 0, 4, 9 and 31 places, and with the upper bits of the shift operand set, which the ALU must ignore. The operand the instruction does not use is set to `0xDEADBEEF`, so that a wrong operand choice shows. `make unit` runs the bench under iverilog in seconds, and passes only on its final `PASS` line, so a bench that stops early fails too.

The bench covers what the golden trace cannot. The golden trace records what the RTL did when it was captured, right or wrong, so a bug present at capture is part of the reference. `SRA` shifted logically when the first trace was captured, and the trace agreed with it until its frame was compared with the C program compiled natively (pitfall P4). The bench holds the shifts to the specification, whatever the trace says. It checks the shifts only; the rest of the ALU is held by the golden trace alone.

9.2 The golden trace

`sim/bench.v` instantiates `SOC` in BRAM boot, with `BOOT_ADDR` set to 0, and records only what an observer outside the chip could see: every byte on the UART line, decoded from the pin as a receiver decodes it, each bit sampled in its middle, and every change of the LEDs, with its time. It reads no internal signal, so the RTL can be restructured freely and the same trace still applies. It writes one line per event: a `UART` line for each byte, a `LEDS` line for each change, with its time in nanoseconds, and an `END` line that says why and when the run stopped. The golden trace opens with ESC and closes with its one LED change:

```
UART 1b
  ⋮
LEDS 1031464038 01
END stop_leds cycles=206292808 uart_bytes=11014 leds=01
```

The run stops when the LED register becomes 1, which the mandelbrot demo writes at the top of its second frame, so the trace holds exactly one frame. The committed trace, `sim/golden/mandelbrot_bram.txt`, is 206 292 808 cycles, 1.031 s of simulated time, with 11 014 UART bytes and one LED change (`docs/baseline.md`). Its `END` line carries the cycle count on purpose: a change that keeps every byte but alters the timing of a single instruction fails, so the trace holds the cycle counts of Table 5.1 as well as the results.

Two comparisons tie the trace to things outside the simulation. Its UART bytes are, byte for byte, the first frame `fw/mandelbrot.c` prints when compiled natively for the PC, so the trace is what the C source means; and they are, byte for byte, the first frame the real board sent, whose time matches the cycle count within the board's timestamps' accuracy, about 2 ms (Chapter 11), so the trace is what the hardware does.

9.3 The regression

`sim/run_regression.sh` compiles the files of `vivado/rtl_sources.txt` and the bench in xsim, links the committed firmware image into its build directory, since `Memory.v` reads it by bare name (pitfall P2), simulates, and compares the new trace with the golden one. It passes only when the two are byte-identical; the full run takes five to six minutes. With `-max-cycles` it is the smoke run: the simulation stops after that many cycles, and the truncated trace must be a prefix of the golden one, which catches most mistakes in 25 seconds. `make golden` overwrites the golden trace with a new run. It is the one way to change the reference, and is meant for an intended change of behaviour only, whose commit says why and which also redoes `make baseline`.

The regression runs one program in one boot mode, so it covers what mandelbrot does in BRAM boot and nothing else. Mandelbrot's code holds 24 of the 37 RV32I instructions other than `FENCE`, `ECALL` and `EBREAK`; it never executes `SLL`, `SRA`, `SLT`, `SLTU`, `XOR`, `SLTI`, `SLTIU`, `XORI`, `ORI`, `LB`, `LH`, `LHU` or `SH`, and no gate runs them through the core; the unit bench checks the ALU's `SRA` alone. The flash is not covered either: the bench has no flash model. Flash boot is simulated only by the report's own bench, `docs/design/scripts/waves/WaveBench.v`, for the waveforms of Figure 6.1, and no gate runs it.

9.4 The gates

The gates are the targets of the root `Makefile` (Table 3.2; Appendix B gives what each runs and needs). Each but `check`, which only runs its three parts, writes its output to a log in `build/logs/`, named after the gate and the time and ending in `PASS` or `FAIL`, and `docs/CONTRIBUTING.md`, "Gates and evidence", says which a change needs: `make lint` before every commit, `make smoke` after each RTL edit, `make -j2 check` once per change, after its review and before any push, and `make baseline` as well when the change must show that timing still holds. A commit message cites the log of each gate it ran. The commit-msg hook in `scripts/hooks` refuses a citation of a log that does not exist or did not pass, and a message that claims a pass without citing a log; the pre-commit hook runs `make lint`, and GitHub Actions runs it again on every push.

`make check` stops after synthesis and reports no timing; only `make baseline` and `make bitstream` route the design.

`make lint` holds what no reviewer should have to check by eye. Rule 1 finds tool by-products beside the sources, which is how a tool run from the wrong directory shows (pitfall P2); rule 2 finds retired paths and vocabulary; rule 3 limits Markdown prose to 100 characters a line; rule 4 holds `vivado/rtl_sources.txt` equal to the files in `rtl/`, since a file missing from it would compile nowhere and fail nowhere; rules 5, 6, 7 and 9 hold the naming and banner conventions of `docs/CONTRIBUTING.md`. Two rules hold copies equal to their source. Rule 8 holds every firmware copy of the memory map, and the README's tables, equal to `rtl/defines.vh` (Section 4.2); rule 10 holds the trace numbers of `docs/baseline.md` equal to the golden trace. Each of the two has a self-test, which changes or deletes every value the rule checks and expects it to fail, and lint runs both self-tests every time.

10 Implementation results

Every number in this chapter is from the routed design that `vivado/baseline.tcl` builds, non-project, for the XC7K325T-2FFG900 with the constraints file's 200 MHz clock, in Vivado 2025.2. `make figures` rebuilds that design in `docs/design/build/` and fails unless its numbers equal those of `docs/baseline.md`, the reference the repository's gates compare against.

10.1 Utilization

Instance	LUTs	as RAM	FFs	BRAM tiles
SOC, the whole design	984	44	244	2
`Processor`, the core	941	44	130	0
its own logic: pc, instruction, address adders	89	0	63	0
`Control`, the FSM	58	0	3	0
`ALU`	2	0	0	0
`RegisterFile`	797	44	64	0
`Memory`, the RAM	0	0	0	2
`IOBus`, the LED register	0	0	8	0
`UartTx`	25	0	26	0
`SPIFlash`	19	0	80	0

Table 10.1: Utilization of the routed design, per instance (`report_utilization -hierarchical`). Each row counts where the cells ended up after synthesis, not where their Verilog is: most of the ALU's logic is counted under `RegisterFile`. A parent's LUTs can be fewer than its children's sum, since a LUT shared across the hierarchy counts once in the parent. The device holds 203 800 LUTs, 407 600 flip-flops and 445 block RAM tiles; the design uses no DSP blocks.

The whole SoC is 984 LUTs and 244 flip-flops, under half a percent of the device, and two of its 445 block RAM tiles (Table 10.1). The core is nearly all of the logic. Inside it, the rows follow the netlist rather than the sources: synthesis optimises across module boundaries, and the ALU's adder, shifter and comparators end up in cells under `cpu/register_file`, leaving 2 LUTs in `ALU`. That is also why the RTL schematic of Figure 5.2 comes from the elaborated design, not the netlist. Of the register file's own LUTs, 44 are distributed RAM: the 32 registers, one copy for each read port. The peripherals are small: `SPIFlash` is mostly flip-flops, its shift registers and counters; `UartTx` is a baud counter and a shift register; `IOBus` is the 8 flip-flops of the LED register; and `Memory` is the two block RAM tiles alone.

10.2 Timing

The one constraint, the 5 ns period of the 200 MHz clock, is met on every one of the 1229 timed endpoints: the worst setup slack (WNS) is 0.219 ns and the worst hold slack 0.113 ns, so the design would close at up to 209.2 MHz.

The worst path (Figure 10.1) starts at `cpu/instr_reg[3]`, an opcode bit of the instruction register, and ends at the data input of the register file's LUT RAM: the write of `rd`. It is the load path. Its first 3 LUTs decode the instruction and choose its immediate; a carry chain of 7 `CARRY4` adds it to `rs1`, the load-store address, as far as bit 24, the bit that selects the flash region (`notes.md`, "Address decode: one bit per region, one-hot IO registers, why bit 24"); and the last 5 LUTs use that bit to choose the bus's read data, cut out the loaded byte or halfword

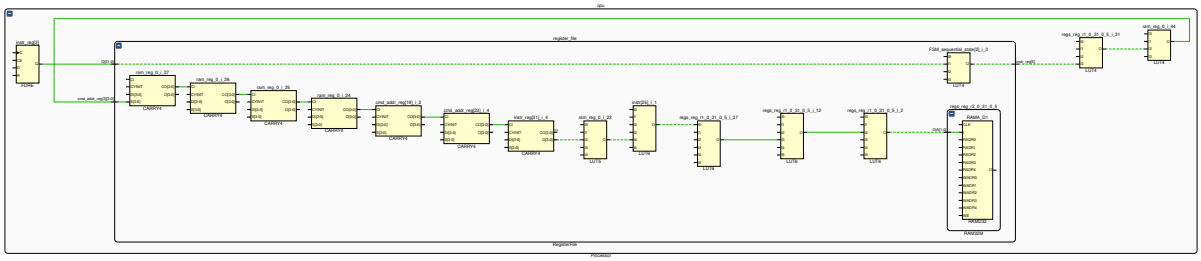
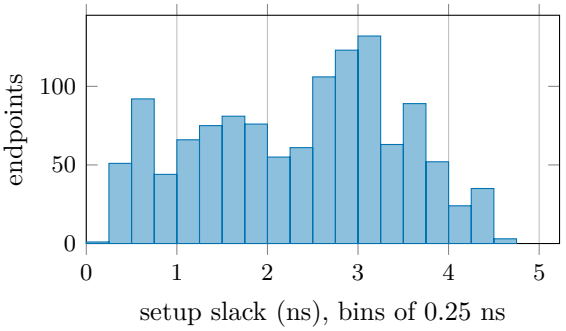
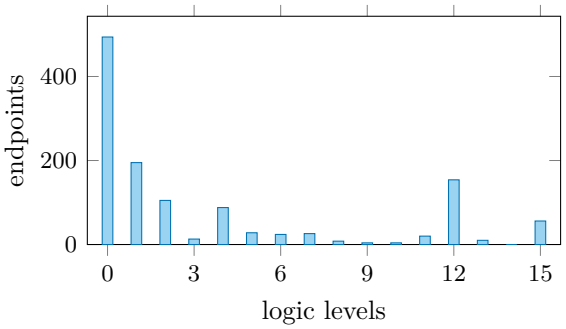


Figure 10.1: The worst setup path in Vivado's schematic, from `instr_reg[3]` at the left to the register file's write data at the right, inside the boxes of `cpu` and `cpu/register_file`: 7 `CARRY4`, 4 `LUT6`, 3 `LUT4`, 1 `LUT5`, between a flip-flop and a LUT RAM. Drawn by `docs/design/scripts/path_schematic.tcl` from the path's pin pairs, so each net shows only its segment on the path.

and pass it to the write port: 15 levels in all. The address is not registered. The core computes it from `instr` and `rs1` in every cycle of a load, so the write in `WAIT_DATA` waits on the whole adder again. Of the path's 4.312 ns, 1.291 ns are in the cells and 3.021 ns, 70 %, in the routing between them, and a clock skew of -0.293 ns counts against it too. Routing dominates, as it usually does on an FPGA: fewer levels, not faster cells, would shorten the path. Registering the load-store address, or only its region bit, would take the adder off it (pitfall P7). Figures 10.4 and 10.5 show it on the device.



(a) Setup slack of each endpoint's worst path. 188 endpoints, 15 %, have less than 1 ns; the most has 4.516 ns.



(b) Logic levels of the same paths. 56 endpoints sit at the deepest, 15 levels.

Figure 10.2: All 1229 timed endpoints, from `get_timing_paths -nworst 1` on the routed design.

Figure 10.2 shows the margin of the rest of the design. The worst path is not alone, but it is not typical either: most endpoints have well over a nanosecond to spare, and 494 endpoints have no logic at all between their two ends.

10.3 Power

Vivado estimates 194 mW on the chip (Figure 10.3). 160 mW of it is static: the leakage of the whole Kintex-7, which a configured FPGA draws whatever the design. The design's dynamic power is 35 mW, from 5 to 9 mW each for the block RAMs, the signals, the logic, the I/O and the clock tree. The estimate assumes every net toggles at Vivado's default rate; with a simulation's switching activity it would be more exact, but not different in kind: the core costs a few milliwatts, and the device costs the rest.

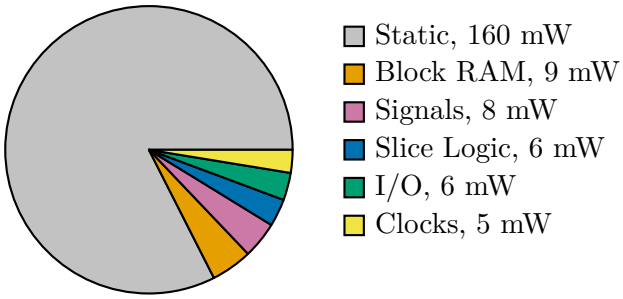
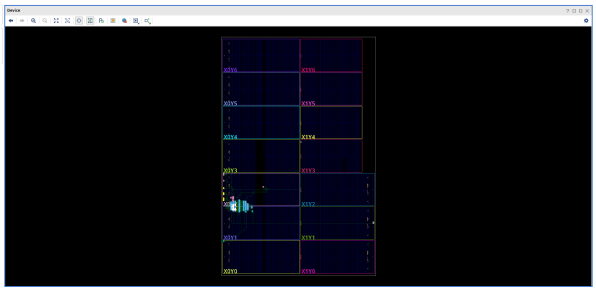


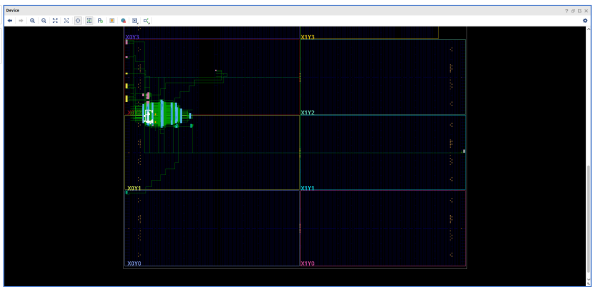
Figure 10.3: On-chip power, 194 mW in all, from `report_power`: a vectorless estimate at Vivado's default toggle rates, with a junction temperature of 25.3 °C. Vivado rounds each figure to the milliwatt, so the parts need not add up to the total.

10.4 The design on the device

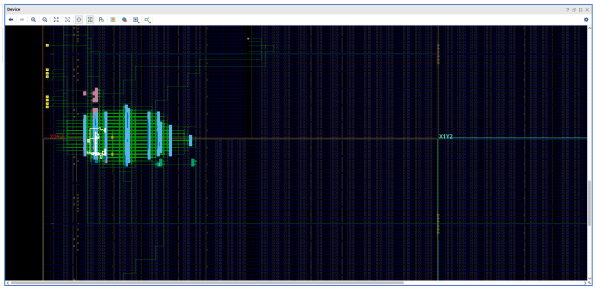
Figures 10.4 and 10.5 are Vivado's Device view of the same routed design, taken by hand in the GUI after `docs/design/scripts/gui_helper.tcl` has coloured the modules and selected the worst path (`figures/README.md` holds the recipe). The whole SoC is a small patch at the left edge of the lower half of the device, with a few nets running out to the pins at its edges; the register file and the rest of the core sit in the middle of the patch, and the worst path stays inside it.



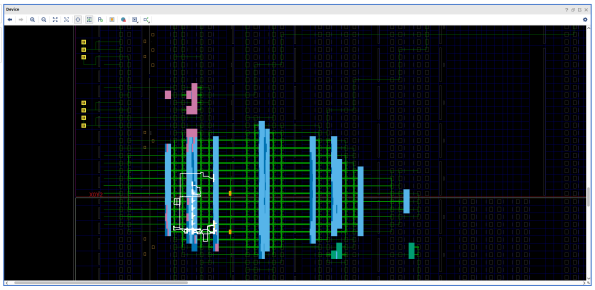
(a) The whole device, its 14 clock regions outlined and named. The design is the coloured patch at the left edge of the lower half.



(b) The lower clock regions. Vivado draws the routed nets in green; some run from the design out to the left and right edges of the device.

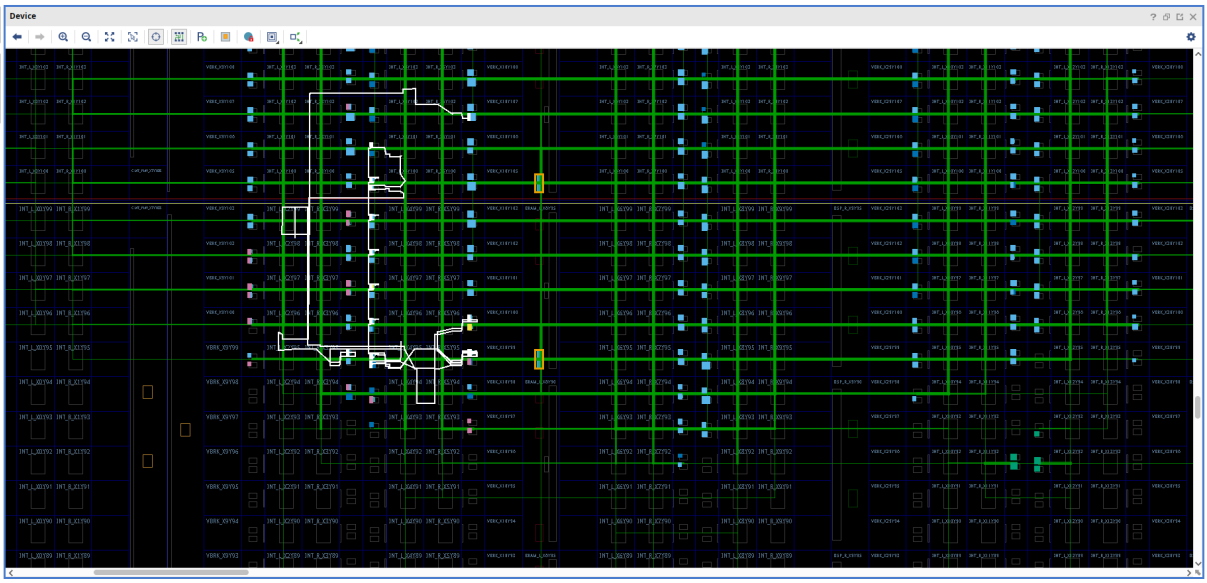


(c) The design: the register file and the rest of the core in the middle, the LED outputs at the left edge, and the worst path in white.

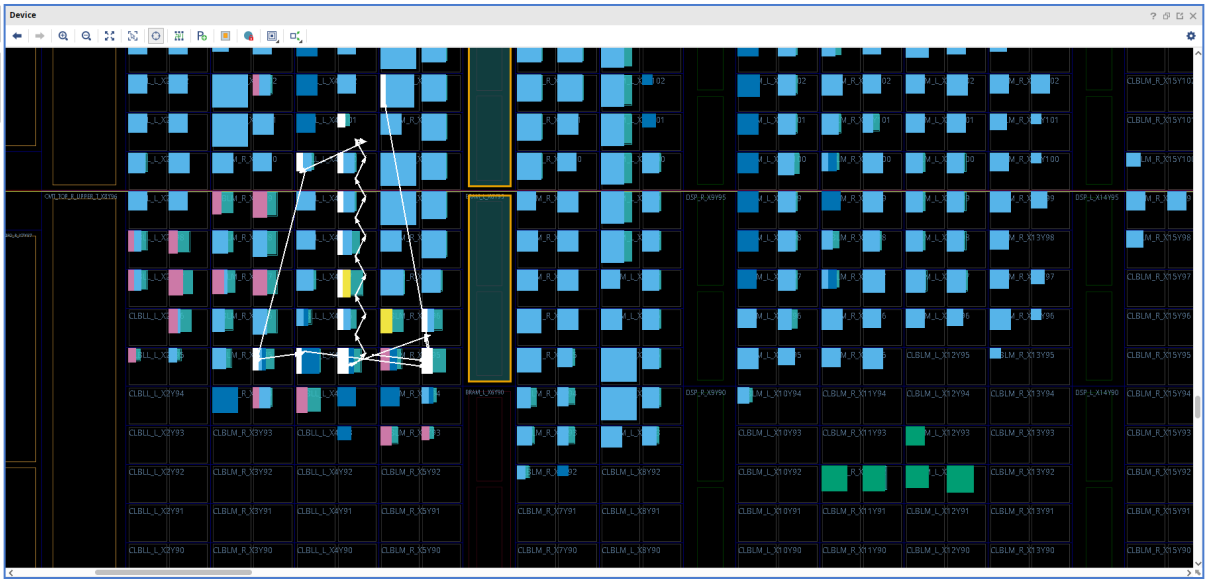


(d) Around the worst path, which ends at a write port of the register file's LUT RAM.

Figure 10.4: Vivado's Device view of the routed design, zoomed in from the whole device to the worst setup path. The modules are in their colours (Table 2.2) and the worst path is selected; Routing Resources is on.



(a) Routing Resources on, its toolbar button pressed: each of the path's nets follows the wires the router chose through the interconnect tiles (`INT_L`, `INT_R`).



(b) Routing Resources off, zoomed further in: each net is drawn as a straight line from its driver to its load, over the slices (`CLBLL`, `CLBLM`) that hold the path's cells.

Figure 10.5: The worst setup path, from `cpu/instr_reg[3]` to the register-file write: 15 logic levels and 0.219 ns of slack at 200 MHz, selected in white.

11 Results on the board

Everything in this chapter was captured from the real board. Each demo's bitstream is loaded into the FPGA over JTAG and never written to the flash, which keeps Digilent's factory bitstream at offset 0: the board boots its demo again at the next power-on. Flash boot writes only the program, at the flash's code offset, over part of the factory demo's data. From the moment the bitstream starts to load, the host records the UART at 115 200 baud, 8N1, with a timestamp on every read, accurate to about 2 ms. The raw captures are committed in `docs/design/captures/`, and `make figures` renders the figures below from them; `docs/design/figures/README.md` gives the procedure.

11.1 Mandelbrot in BRAM boot

The mandelbrot demo is the committed firmware image, the one the golden trace was simulated with. Its bitstream is the synthesised design with `BOOT_ADDR` set to `BOOT_ADDR_BRAM`, as the testbench sets it, so the core starts from the image in the RAM as soon as configuration ends. Figure 1.1, which opens this report, is the first frame the board sent.

The board against the simulation. The first frame the board sent is 11 014 bytes long, and it is byte for byte the UART bytes of the golden trace. The golden trace is in turn byte for byte what `fw/mandelbrot.c` prints when compiled natively (`docs/baseline.md`), so the board, the simulation and the C source agree.

Frame time. On the board the frame took 1031.8 ms, from its first byte to the first byte of the next frame. The simulation took 206 292 808 cycles from reset to the start of the next frame, 1031.46 ms at 200 MHz. The start-up code before the first byte is a few dozen instructions, so the two measure the same interval. They agree to 0.3 ms, within the timestamps' accuracy of about 2 ms, or 0.2 %: to that accuracy, the board takes the cycles the simulation counts, at 200 MHz. Most of the frame is the UART. At ten bits per byte, sending the 11 014 bytes takes about 956 ms, and `putchar` waits for each byte to leave before it returns, so the computation itself takes about 75 ms.

11.2 riscv_logo

The riscv_logo demo rotates and zooms the RISC-V logo, a 16 by 16 bitmap, tiled over a grid of 40 by 40 pixels in the 16 standard terminal colours, here in xterm's palette. One turn is 64 frames. The board sent them in 35.3 s, 551 ms a frame on average and 388 to 675 ms each: a frame is 4 400 to 7 700 bytes, and the UART sets the pace, as for mandelbrot. Figure 11.1 shows every eighth frame, and Figure 11.2 the whole turn. The 72 frames captured are, byte for byte, what `fw/riscv_logo.c` prints when compiled natively: no byte was lost on the way to the host.

11.3 tinyraytracer in flash boot

tinyraytracer is 29 016 bytes of code and constants, almost five times the 6 KB RAM, so it runs only in flash boot, executed in place from the flash (Chapter 7). Its flash image was written to the code window at `0xD00000` with "Configuration File Only". A readback of the whole 32 MB chip afterwards differed from the one taken before only in the image's own 29 016 bytes: the factory bitstream at offset 0 was not touched. Before each capture, the capture script reads the code window back and compares it with the binary.

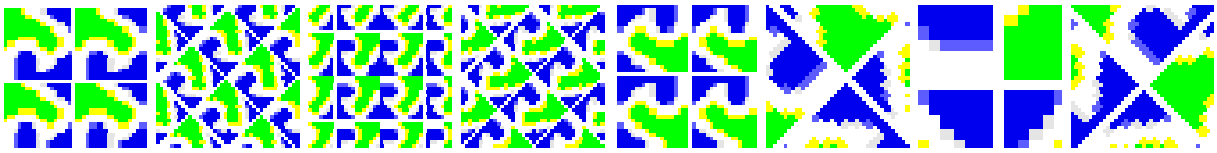


Figure 11.1: The riscv_logo demo on the board: frames 0, 8, ..., 56 of one turn, left to right, 45 degrees apart, rendered from the UART capture. The animation plays in the repository's README, at `https://github.com/amitmalach/FPGA_RiscV_MultiCycle#running-the-demos`, the address the code on the right holds.

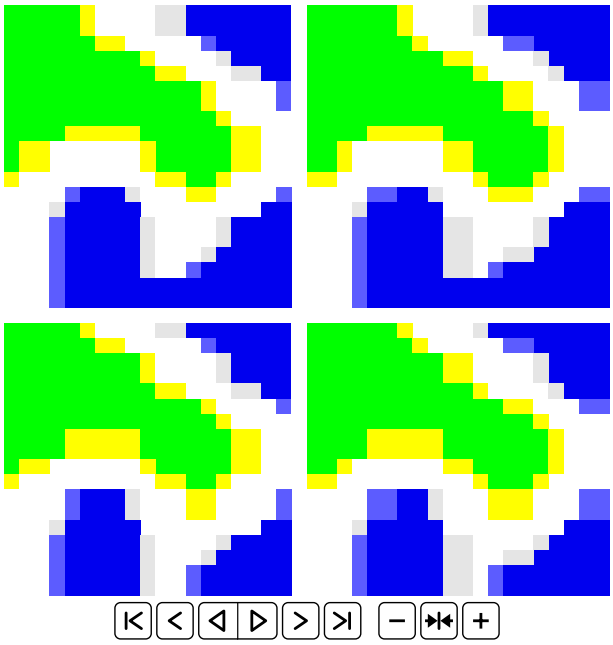


Figure 11.2: The riscv_logo demo's turn, all 64 frames, at the board's average rate of 1.81 frames per second. Only Adobe Reader plays the animation; other viewers show the first frame.

The reset button. With the flash-boot bitstream loaded over JTAG, the core printed nothing and the LEDs stayed dark: in the capture behind Figure 11.3, no byte arrived in the 44 s between the end of the load and a press of the CPU reset button. From the press on, the demo ran to the end, so flash boot works on this board with the reset button pressed once after each load (pitfall P5).

The Quad Enable bit. Writing the image cost the board its power-on boot: the write cleared the flash's non-volatile Quad Enable bit, which the factory bitstream needs, and at the next power-on the FPGA stalled in configuration with every byte of the flash intact. A small design loaded over JTAG set the bit again, a further write of the image with `PROGRAM.SKIP_QE_BIT_ERASE` set kept it, and at the next power-on the board booted Digilent's factory demo again. Pitfall P8 gives the cause and the tools.

The picture. Figure 11.3 shows what the board sent: 48 by 24 pixels, each one terminal cell, a space on one of xterm's 256 colours. The run is 21 412 bytes long and is, byte for byte, what `fw/tinyraytracer.c` prints when compiled natively, with `libm` and floating point in software on the board and in the host's hardware on the PC. The picture took 126.8 s, 110 ms a pixel: every floating-point operation is a call into `libgcc`, and every instruction outside the RAM is

fetched from the flash, one SPI transaction each.

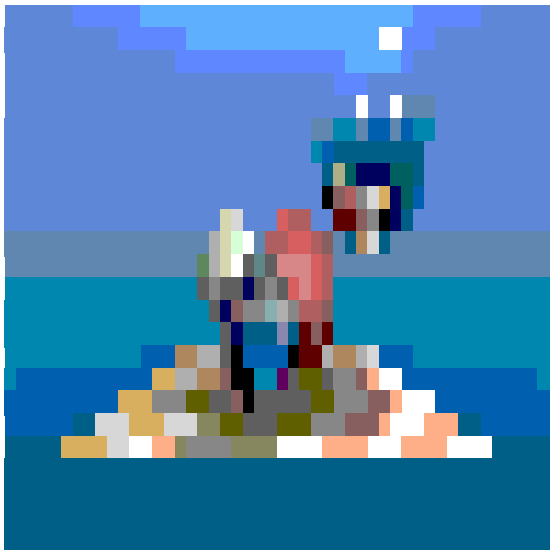


Figure 11.3: tinyraytracer in flash boot, rendered from the board's UART capture: four spheres, of ivory, glass, red rubber and mirror, over a chequered floor and lit by three lights, at 48 by 24 terminal cells.

11.4 ST_NICCC

ST_NICCC was not run. It needs the 640 KB scene data at `0xC00000` and its own program in the code window, two more writes over the factory demo's data, and the reset button as above.

11.5 blinker and sieve

Both ran in BRAM boot. The blinker alternates LEDs 0 and 2 with LEDs 1 and 3 every 5.24 ms in simulation, about 95 times a second: too fast to see, and on the board LD0 to LD3 glowed steadily at about half brightness while LD4 to LD7 stayed dark. sieve printed the 31 primes below 128 and its checksum, `1772A48F OK`, every 96 ms; the 59 rounds captured are, byte for byte, what `fw/sieve.c` prints when compiled natively, but for the checksum's hex digits, which `fw/print.c` prints in upper case.

12 For the next student

This chapter is for whoever extends the design next: the traps this project fell into, and where to go from here.

12.1 Pitfalls

Each pitfall below cost time once. Each gives the symptom as it showed, or would show, the cause, and where the design or the repository now handles it; the other chapters cite them by number.

P1 The UART pin-direction trap

Symptom. Not one byte reaches the PC and the bridge's activity LED, LD12, never blinks, while the design looks healthy inside, in simulation as well.

Cause. The board's net names mix two points of view: TX and RX are the PC's, `_in` and `_out` the FPGA's. `uart_rx_out` is the FPGA's transmit line. Reading `_in` as "into the PC" swaps the two pins, and the transmitter drives Y20 against the bridge's own TXD. A testbench that prints what the core handed the UART, instead of decoding the pin, cannot see it.

Handled. The ports of `SOC` keep their names and directions, which the constraints file and the testbench bind to (`docs/CONTRIBUTING.md`), and `sim/bench.v` decodes the `uart_rx_out` pin as a receiver does. Chapter 2 follows the nets through the schematic (`notes.md`, "UART port names and the pin-direction trap").

P2 The firmware image read by bare file name

Symptom. In simulation the program does nothing: the LEDs stay at zero and the RAM reads `x`, with only a warning in xsim's log. After a Vivado run, `.Xil/` and `clockInfo.txt` appear beside the sources.

Cause. `Memory.v` reads `firmware.hex` with `$readmemh` by bare name, relative to the working directory of whichever tool runs, and a missing file leaves the RAM undefined without an error. A tool started in the wrong directory either finds no image or leaves its by-products there.

Handled. Every script that simulates or synthesises runs in a directory of its own with the image linked or copied in: `sim/run_regression.sh` links it into `sim/build/`, the root `Makefile` starts Vivado in `build/`, and the Vivado project lists the image in both its filesets. Lint rule 1 fails on by-products beside the sources (Chapter 9), and the header of `rtl/Memory.v` warns (`notes.md`, "Bare file name in $readmemh").

P3 The simulation hang without initial values

Symptom. The simulation runs but the flash controller never starts its clock, and the testbench hangs; the same RTL works on the board.

Cause. A register with no initial value starts as `x` in simulation, and `x + 1` stays `x`: the clock divider of `SPIFlash` never counted. Configuration loads a definite value into every flip-flop of the FPGA, so the silicon never shows the problem.

Handled. `div` and `sclk` in `rtl/SPIFlash.v` declare initial values (`notes.md`, "Explicit initial values on div and sclk (simulation hang)"), and the core's registers declare their reset values as initial values (Section 4.3). The register file and the RAM declare none, hence the `x` in Figure 5.3. A new counter needs an initial value too.

P4 SRA behaving as SRL

Symptom. The golden trace's frame diverged from the C program compiled natively at row 8, pixel 36, and held 8 368 bytes instead of 11 014: `-1544416 » 9` gave `0x007FF437` where `0xFFFFF437` was right.

Cause. In Verilog an expression's operands decide its signedness together. In `funct7[5] ? ($signed(a) >> s) : (a >> s)` the unsigned arm makes the whole conditional unsigned, and `>>` then shifts logically.

Handled. Both arms of the shift in `rtl/ALU.v` are signed; `sim/ALUBench.v` checks `SRA`, `SRAI`, `SRL` and `SRLI` on negative operands (Chapter 9); the golden trace was captured again. The sign fill cost 54 LUTs, 930 to 984 (`docs/baseline.md`) (`notes.md`, "SRL versus SRA, as the comment described it").

P5 Flash boot loses its first fetch

Symptom. With the flash-boot bitstream loaded over JTAG, no byte arrives on the UART and the LEDs stay dark. One press of the CPU RESET button, and the demo runs from then on (Chapter 11).

Cause. Most likely the core's first fetch from the flash is lost straight after configuration. That fetch is `lui gp,0x400`, the first instruction of `fw/start_spiflash.S`, which sets `gp` to `IO_BASE`; without it the UART writes, at `gp + 8`, land in RAM address 8, over the code start-up has just copied there. Two parts of the design can lose it. `STARTUPE2` passes `USRCCLKO` to the CCLK pin only after a few edges, so the first transaction misses clock edges and the flash misreads its command; in simulation this alone loses the fetch (Section 7.3). And `IOBus` gates `spiflash_rstrb` with `eos`, so a fetch requested before end of startup is dropped instead of stalled.

Handled. On the board only: press the reset button once after each load (the README's "Flash boot"). The RTL is unchanged in this report. A candidate fix, not kept in the repository, made the board print straight after the load: `eos` moves from `IOBus` into `SPIFlash`, which makes one throwaway read as soon as `eos` rises, and holds a read requested before that read is done, busy, until it can start it. The throwaway read's address is all ones, so a frame that loses its first clock edges shows the flash only shifted read opcodes, never a write enable, a program or an erase. An implementation run that was most likely of this fix used about 25 LUTs and 25 flip-flops more than the committed design, and met timing with 0.334 ns of slack.

P6 Reset landing in WAIT_DATA

Symptom. With the reset state changed to `FETCH_INSTR`, the obvious choice, nothing shows in BRAM boot, where the regression runs; in flash boot the first fetch after a press of the reset button would go wrong.

Cause. `FETCH_INSTR` asserts the read strobe for as long as it lasts. Held in reset there, with `pc` in flash space, the core would restart the SPI transaction on every cycle, chip select held low: the flash would see a command it never finishes, and no deselect to resynchronise on.

Handled. `rtl/Control.v` resets into `WAIT_DATA`, which asserts nothing and falls through to `FETCH_INSTR` in the first cycle after reset (Section 4.3). Its write-back is harmless, because the instruction register resets to a no-op whose destination is `x0` (`notes.md`, "Reset lands in WAIT_DATA, not FETCH_INSTR").

P7 The worst path ends in the register-file write

Symptom. A small change on the load path, one more term in the address decode of `IOBus` or in the load lanes, can miss the 5 ns of the 200 MHz clock: the worst path has 0.219 ns of slack.

Cause. The load-store address, `rs1 + imm`, is not registered, so every cycle of a load computes it again. The path runs from `instr_reg[3]`, an opcode bit that picks the immediate, through 7 `CARRY4` of the adder to bit 24, the region decode, the read-data multiplexer and the load lanes, into the register file's write data: 15 levels and 4.312 ns, 3.021 ns of it routing (Chapter 10).

Handled. Not yet: a comment at `loadstore_addr` in `rtl/Processor.v` marks it. Register the address in `EXECUTE`, or only its region bit, and the adder leaves the path. `LOAD` and `STORE` follow `EXECUTE`, so the register costs no cycle.

P8 Writing the flash clears the Quad Enable bit

Symptom. After a flash write with "Configuration File Only", the board no longer boots at power-on: PROG lit, DONE off, the OLED blank, no CRC, ID or bad-packet error, the mode pins right, and every byte of the flash intact.

Cause. Vivado's `program_hw_cfgmem`, with `PROGRAM.SKIP_QE_BIT_ERASE` at its default, 0, clears the S25FL256S's non-volatile Quad Enable bit, bit 1 of configuration register 1. The factory bitstream reads the flash four bits wide (opcode `0x6C` in its BSPI register) and needs it. A write with an SPIx4 `.prm` does not set it back: Vivado's width-specific part names for this chip are for Zynq only, and on a Kintex-7 it accepts only `s25fl256sxxxxxx0-spi-x1_x2_x4`. Vivado's Tcl has no command that reads the flash's registers [6].

Handled. In `docs/design/scripts/`, `board_flash.tcl write` sets `SKIP_QE_BIT_ERASE` to 1, and a write with it kept the bit; `flashregs/FlashRegs.v`, loaded over JTAG, prints SR1, CR1, the chip's ID and the sync word at `0x30` on the UART; built with `WRITE_REGS=1` it writes CR1 back to `0x02`, which repaired the owner's board ("The Quad Enable bit" in `docs/design/figures/README.md`).

P9 The stack runs into the program

Symptom. A program that has grown wanders off into garbage after a while, or prints nonsense, with no warning from the build.

Cause. In BRAM boot the 6 KB of RAM hold the code, the data and the stack, and the stack grows down from `0x1800` into the same space; in flash boot it shares the RAM with the copied code (Chapter 7). Nothing checks how deep the stack goes.

Handled. In part. `firmware_words` prints how much of the RAM the image fills and refuses one past `-max_addr`, which `fw/Makefile` sets to the RAM size; set lower, it reserves room for the stack (`notes.md`, "firmware_words arguments; -max_addr as a stack guard"). The stack itself is not checked.

P10 Debug data loaded into the RAM

Symptom. None at run time: mandelbrot runs as it should. The image is larger than its code, though: `firmware_words` reports 852 words, 3 408 bytes, and `objdump -D` shows runs of `.insn` nonsense after `__mulsi3` and after `__modsi3`.

Cause. `fw/bram.ld` keeps the catch-all `*(.*)` of [1], which places every section of every input file in the RAM. Ubuntu's `libgcc.a` carries DWARF debug sections in its members, so those of `muldi3.o` and `div.o` land right after their code: 1 084 bytes, almost a third of the image and 18 % of the 6 KB, never fetched but taken from the room the stack grows into (pitfall P9). The `/DISCARD/` line drops only `.riscv.attributes` and `.comment`, which caused the same trouble before. `fw/spiflash.ld` names its input sections and is not affected.

Handled. Not handled; the committed image is the one this report describes. Adding `*(.debug*)` to `/DISCARD/` gives a 2 324-byte image; only `div.o`'s routines move, `__divsi3` from `0xaa0` to `0x860`, and the calls to them with it (`notes.md`, "bram.ld line by line; the /DISCARD/ workaround").

12.2 Extensions, ranked

The list is ranked by what each extension gains for the work it takes, the ones others build on first. None is begun; each names where it enters the design and which pitfalls lie on the way.

1. **Pipelining.** An instruction takes four to six cycles (Table 5.1); a five-stage pipeline retires close to one a cycle, a gain of four to six on code bound by computation. It needs a second memory port, which the block RAM has, forwarding and a stall after a load, a flush on a taken branch, and a stall of the whole pipeline while `mem_rbusy` is high. The stages split the worst path (pitfall P7). The golden trace's cycle count will change, and its bytes must not:

a new golden trace, with `make golden` and the reason. In flash boot every fetch costs 382 cycles, so there a small instruction cache in front of `SPIFlash` gains more than a pipeline.

2. **The M extension.** Every `*`, `/` and `%` in C is a call into `libgcc` today (Section 7.4), and mandelbrot multiplies in its inner loop. The device's DSP blocks, none of them used, multiply 32 by 32 bits in a few cycles, and a divider fits the FSM as one more state that holds like the waits. The firmware then builds with `-march=rv32im`.

3. **UART receive.** `uart_tx_in`, pin Y20, is wired and unused. A receiver mirrors `UartTx`, and a data register and a ready bit take the next IO address bit, bit 5 (Section 4.2), with their firmware copies added to lint rule 8. It allows a serial boot loader: a new program in the RAM without a new bitstream or a flash write. Mind the pin names (pitfall P1).

4. **Interrupts.** They need the control and status registers of Zicsr, at least `mstatus`, `mtvec`, `mepc` and `mcause`; today their opcode stops the core (Appendix C). A timer and the UART receiver are the natural first sources. They touch `Decoder`, `Control`, which needs a trap state, and the `pc` multiplexer of `Processor`.

5. **DDR3.** The board's 1 GB comes through AMD's Memory Interface Generator, with a clock domain of its own and a latency of tens of cycles. The bus already waits on reads, through `mem_rbusy`, but has no way to stall a write, and without a cache every access pays the latency.

6. **HDMI.** A pixel clock, TMDS encoding through the output serialisers, and a frame buffer: 300 KB for 640 by 480 pixels at 8 bits, which fits in the device's 2 MB of block RAM but belongs in DDR3, which is why it comes last.

13 Conclusions

The project set out to run compiled C on a processor of its own, on a real board, and to understand every layer in between. It did. A multi-cycle RV32I core of 984 LUTs, with 6 KB of RAM, three IO registers and a flash controller, runs at the Genesys 2's 200 MHz with 0.219 ns to spare, draws the Mandelbrot set over the UART from the block RAM, and renders a ray-traced picture in place from the flash. Every stage from the C source to the UART bytes is shown with a real excerpt, and every figure a script can make is made by one.

Three things the results show are worth stating plainly. First, the simulation is a faithful stand-in for the board: the frame the board sent is byte for byte the golden trace's, and its time matches the trace's cycle count to a fraction of a percent, so a change that keeps the trace keeps the hardware's behaviour, and the RTL can be restructured against the trace alone. Second, the core is not what limits these demos; the UART is. A byte takes 87 µs, and a frame is nearly all bytes, so a pipelined core or a multiplier would change nothing visible until the output path changes with it. Third, most of the time lost went to things outside the core: the board's net names, the working directory a tool runs in, Verilog's signedness rules, the flash clock's hand-over after configuration, and a bit in the flash's configuration register. Chapter 12 records each of them with its symptom, so that the next student loses hours where this project lost days.

What is left is also recorded. Flash boot needs a press of the reset button after each load (pitfall P5), and a candidate fix is described but not kept; the stack is unguarded (pitfall P9), and the firmware image carries debug data it never fetches (pitfall P10); thirteen of RV32I's instructions are executed by no gate; and ST_NICCC, the demo that would end the report on its most striking picture, waits for two flash writes. Beyond those, Chapter 12 ranks the extensions: pipelining first, then the M extension, UART receive, interrupts, DDR3 and HDMI, each with the part of the design it enters and the pitfall that lies on its way. The board has room for all of them.

A Memory map

Table A.1 gives every memory-map constant of `rtl/defines.vh`, the one place the map is defined, and Table A.2 the decode `IOBus` makes of an address. The firmware's copies of these values, in `fw/io.h`, `fw/putchar.S`, `fw/blinker.S`, `fw/start*.S`, `fw/memory.ld` and `fw/Makefile`, are held equal to them by lint rule 8.

Constant	Value	Meaning
`RAM_BASE`	`0x0000_0000`	the RAM: no region bit set
`RAM_WORDS`	1536	RAM depth in words, 6 KB, up to `0x0000_17FF`
`IO_BASE`	`0x0040_0000`	the IO registers: bit 22
`IO_LEDS`	`0x04`	write: the LED register, bit 2
`IO_UART_DAT`	`0x08`	write: send one byte, bit 3
`IO_UART_CNTL`	`0x10`	read: the UART status, bit 4
`UART_BUSY_BIT`	9	status bit: the transmitter is busy
`SPIFLASH_BASE`	`0x0100_0000`	the SPI flash: bit 24
`SPIFLASH_ADDR_W`	24	address bits inside the flash: 16 MB
`SPIFLASH_CODE_OFFSET`	`0x00D0_0000`	the code window in the chip, 13 MB in
`BOOT_ADDR_FLASH`	`0x01D0_0000`	flash boot's reset vector
`BOOT_ADDR_BRAM`	`0x0000_0000`	BRAM boot's reset vector

Table A.1: The memory-map constants of `rtl/defines.vh`. IO offsets are from `IO_BASE`.

Address	Decoded as
bit 24 set	Flash, whatever the other bits: byte offset `addr[23:0]`, word `addr[23:2]`. Reads and fetches stall for one SPI transaction, once end of startup has passed (before it, pitfall P5); writes do nothing
bit 24 clear, bit 22 set	IO. A store writes the LED register if bit 2 is set and sends a byte if bit 3 is set, both at once if both are; a load returns the status word if bit 4 is set, else zero
neither	RAM, word `addr[31:2]`. Above `0x0000_17FF` the access is undefined; in simulation it reads `x`

Table A.2: The address decode of `rtl/IOBus.v`: one bit tested per region and per IO register, no address refused (Section 4.2).

The flash chip itself (Figure 7.1) holds the bitstream from `0x000000` to `0xAE9E0F`, ST_NICCC's scene data from `0xC00000`, and the code window from `0xD00000` to `0xFFFFFF`; the core cannot reach the chip's upper 16 MB. In both boot modes the stack starts at the top of the RAM, `0x1800`, and grows down (pitfall P9).

B Make targets

Four Makefiles, each documented in its header. The root one holds the gates (Chapter 9); Tables B.1 and B.2 list every target of all four.

Target	What it does	Needs
Root `Makefile`: the gates, each but `check` with a log in `build/logs/`		
`lint`	`scripts/lint.sh`, seconds	bash, git
`unit`	The ALU unit bench, seconds	iverilog
`check-fw`	`make -C fw check`	RISC-V gcc
`smoke`	`unit`, `check-fw`, 4 million cycles of regression, about 25 s	all of the above, Vivado
`check-sim`	`check-fw`, the full regression, five to six minutes	gcc, Vivado
`check-synth`	`check-fw`, synthesis in project mode, under a minute	gcc, Vivado
`check`	`unit`, `check-sim`, `check-synth`; `-j2` runs the last two side by side	iverilog, gcc, Vivado
`golden`	`check-fw`, a full run that overwrites the golden trace	gcc, Vivado
`check-moves BASE=`*rev*	Moved files are byte-identical renames	git
`bitstream`	The project's synthesis, implementation and `build/SOC.bit`, with WNS, WHS and `timing=MET` or `FAILED`	Vivado
`baseline`	Non-project implementation and reports into `build/baseline/`, about a minute	Vivado
`fw/Makefile`; `PROG` picks the demo, `TARGET` the boot mode		
`all` (default)	`bram` or `flash`, as `TARGET` says	RISC-V gcc
`bram`	The demo's image, written to `HEX_OUT`, by default `rtl/firmware.hex`	gcc, g++
`flash`	`build/`*prog*`.spiflash.{elf,bin,mcs}`, the code at `0xD00000`	gcc, Vivado
`build/`*prog*`.s`	The demo's assembly, for the reader	gcc
`check`	The memory-map copies, then the image rebuilt from clean and compared	gcc, g++
`print-hex-path`, `clean`	Print `HEX_OUT`; remove `build/`	

Table B.1: The make targets of the root `Makefile` and of `fw/Makefile`, which also takes `BUILD`, `HEX_OUT`, `RAM_SIZE`, `CROSS` and `XILINX_RV32_ROOT`.

Target	What it does	Needs
docs/design/Makefile: this report		
`all` (default)	`build/design.pdf`, from the committed figures	pdfLaTeX
`figures`	Every scriptable figure: the five below and the RTL schematics	Vivado, xvfb-run, and the below
`board-figures`	The figures of the board captures	Python, Pillow
`waves`	The three waveforms	iverilog, gcc, Vivado's models, Ghostscript
`flow`	The excerpts of the C-to-board figure	gcc, Vivado, Pillow
`impl`	Chapter 10's numbers and worst-path schematic	Vivado, xvfb-run, Ghostscript
`schematics`	The excerpts of the board schematic, from a local copy	pdftoppm
`captures`	The BRAM-boot board captures, recorded again	the board, Vivado, gcc, pyserial
`clean`	Remove `build/`	
sim/asm/Makefile		
`program.hex` (default)	The sample program, assembled by the Verilog assembler; `DEFS=-DBENCH` for full speed in simulation	iverilog
`clean`	Remove its outputs	

Table B.2: The make targets of `docs/design/Makefile`, which also takes `SCHEMATIC`, the path of the board schematic, and of `sim/asm/Makefile`.

C ISA coverage

The core executes the RV32I base integer instruction set [2], and the firmware is built with `-march=rv32i -mabi=ilp32`, so the compiler emits nothing else. Table C.1 gives each instruction of RV32I and what the core does with it.

Class	Instructions	In this core
Register	`ADD SUB SLL SLT SLTU XOR SRL SRA OR AND`	executed, 4 cycles
Immediate	`ADDI SLTI SLTIU XORI ORI ANDI SLLI SRLI SRAI`	executed, 4 cycles
Upper immediate	`LUI AUIPC`	executed, 4 cycles
Jump	`JAL JALR`	executed, 4 cycles
Branch	`BEQ BNE BLT BGE BLTU BGEU`	executed, 4 cycles
Load	`LB LH LW LBU LHU`	executed, 6 cycles
Store	`SB SH SW`	executed, 5 cycles
Fence	`FENCE`	a no-op, without being decoded
System	`ECALL EBREAK`	stop the core

Table C.1: RV32I in this core. The cycles are from the RAM (Table 5.1); a word fetched or loaded from the flash adds an SPI transaction.

FENCE. Its opcode sets none of `Decoder`'s class flags, so the core treats it as an ALU instruction: `EXECUTE` advances `pc` and writes the ALU's output to the instruction's `rd` field, which is `x0` in a `FENCE`. With one core, no cache and memory accessed in program order, a no-op is all a `FENCE` has to do. `FENCE.I` shares the opcode, and is a no-op too, also correctly: instructions and data come from the same memory, uncached.

ECALL and EBREAK. `EXECUTE` leaves `pc` unchanged for the SYSTEM opcode, so the core fetches the instruction again, forever: a halt that the reset button ends. The start-up code ends with `EBREAK`, should `main` return.

Left out. No extension: no M, A, F, D or C, and no Zicsr. The CSR instructions share the SYSTEM opcode and stop the core as `ECALL` does, though one whose `rd` is not `x0` also writes the ALU's output to it on every pass. No privilege modes, no exceptions and no interrupts: an illegal instruction is executed as `FENCE` is, its ALU output written to its `rd` field, and a misaligned load or store is not trapped either. It takes its lanes from the aligned word around its address, so its result is wrong, though code built from C never makes one for aligned data.

Bibliography

[1] B. Levy, *learn-fpga*, "From Blinker to RISC-V", GitHub repository. `https://github.com/BrunoLevy/learn-fpga`

[2] A. Waterman and K. Asanović (eds.), *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, document version 20191213, RISC-V Foundation.

[3] Digilent, *Genesys 2 Reference Manual*. `https://digilent.com/reference/programmable-logic/genesys-2/reference-manual`

[4] Digilent, *Genesys 2 Schematic*, revision H.1, 22 pages, 2015, in the Genesys 2 Resource Center. `https://digilent.com/reference/programmable-logic/genesys-2/start`

[5] AMD Xilinx, *7 Series FPGAs Configuration User Guide*, UG470.

[6] AMD Xilinx, *Vivado Design Suite Tcl Command Reference Guide*, UG835.